

Lab Manual

Database Systems

CS-213



By

Humna Imran

2023-BSAI-017

Submitted to

Mr. M. Ejaz Bashir

Bachelor of Science in Artificial Intelligence

DEPARTMENT OF COMPUTER SCIENCES

Table of Contents

Lab No 1: Creating Table Using Oracle User Interface	3
Lab No 2: Creating Tables Using Ddl Queries In Sql	6
Lab 3: Insert, Update, Delete Table Data Using Dml	13
Lab 4: The Practice Of Ddl And Dml Queries With Different Case Scenarios	18
Lab 5: Where And Order By Clause	30
Lab 6: Other Restricting Data Constraints	41
Lab 7: Practical Test (Create, Alter, Drop, Insert, Update, Delete)	48
Lab 8: Database Setup Using Xampp	53
Lab 9: Mysql Ddl Queries Using Different Scenarios	56
Lab 10: Mysql Dml Queries Using Different Scenarios	58
Lab 12: Join (Cross, Inner And Outer Join (Left, Right, Full)	78
Lab No 13: Self-Join, Multi Join, And Union	85
Lab No 14: Sql Functions (Math And String)	90
Lab 15: Date & Aggregate Function	101

LAB NO 1: CREATING TABLE USING ORACLE USER INTERFACE

Create Table:



Figure 1: Creating a table by selecting "Object Browser" > "Create" > "Table"

Add Columns:

The screenshot shows the 'Create Table' dialog box. On the left, there is a sidebar with buttons: Columns, Primary Key, Foreign Key, Constraints, and Confirm. The main area of the dialog box has the following fields and options:

- Table Name:** STUDENT
- ☐ Preserve Case

Column Name	Type	Precision	Scale	Not Null	Move
REGISTRATION	VARCHAR2		4000	☐	▼ ▲
NAME	VARCHAR2		4000	☐	▼ ▲
DOB	DATE			☐	▼ ▲
PHONE	VARCHAR2		4000	☐	▼ ▲
CNIC	VARCHAR2		4000	☐	▼ ▲
	- Select Datatype -				▼ ▲
	- Select Datatype -				▼ ▲
	- Select Datatype -				▼ ▲

At the bottom right of the dialog box, there is an 'Add Column' button.

Figure 2: Creating the "Students" entity with attributes: Registration Number, Name, Date of Birth, and Phone/Cell Number.

Primary Key:

Columns
▼
Primary Key
▼
Foreign Key
▼
Constraints
▼
Confirm

Primary Key

Cancel
< Previous
Next >

Table name: **STUDENT**
Primary Key: ☒ No Primary Key
☐ Populated from a new sequence
☐ Populated from an existing sequence
☐ Not populated

Figure 3: Primary Key Selection for STUDENT Table

Foreign Key:

Columns
▼
Primary Key
▼
Foreign Key
▼
Constraints
▼
Confirm

Foreign Keys

Cancel
< Previous
Next >

Foreign Key	Columns	Referenced Table	Referenced Columns	Action
Add Foreign Key * Name STUDENT_fk ☒ Disallow Delete ☐ Cascade Delete ☐ Set Null on Delete Select Key Column(s) REGISTRATION NAME DOB PHONE CNIC * Key Column(s) * References Table Add				

Figure 4: Foreign Key Configuration for STUDENT Table

Constraints:

Columns
▼
Primary Key
▼
Foreign Key
▼
Constraints
▼
Confirm

Constraints

Cancel
< Previous
Finish

Constraint Name	Type	Column(S)/Check
Add Constraint ☒ Check ☐ Unique * Name STUDENT_ck1 > Available Columns > Example Check Constraints		

Figure 5: Constraints Configuration for STUDENT Table

Confirm:

Columns
▼
Primary Key
▼
Foreign Key
▼
Constraints
▼
Confirm

Create Table

Cancel Create

 Please confirm your request.

Schema: **HR**
Table name: **STUDENT**

 SQL

Figure 6: Table Creation Confirmation for STUDENT Table

Table Schema:

Table	Data	Indexes	Model	Constraints	Grants	Statistics	UI Defaults	Triggers	Dependencies	SQL
Add Column Modify Column Rename Column Drop Column Rename Copy Drop Truncate Create Lookup Table										
Column Name	Data Type	Nullable	Default	Primary Key						
REGISTRATION	VARCHAR2(4000)	Yes	-	-						
NAME	VARCHAR2(4000)	Yes	-	-						
DOB	DATE	Yes	-	-						
PHONE	VARCHAR2(4000)	Yes	-	-						
CNIC	VARCHAR2(4000)	Yes	-	-						
1 - 5										

Figure 7: Table Structure of STUDENT Table

LAB NO 2: CREATING TABLES USING DDL QUERIES IN SQL

1. Create Table Using SQL Command:

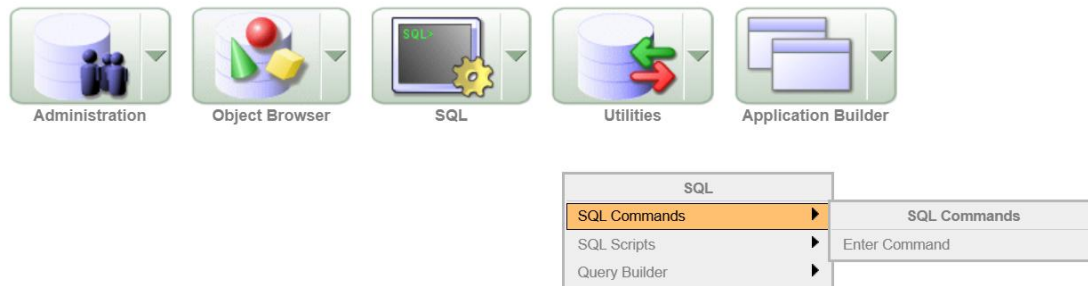


Figure 8: SQL Command Interface in Database Management System

Enter Commands:



Figure 9: SQL Commands Interface

1.1. Task No 1:

Query:

```
CREATE TABLE "TUF_STUDENTS"
("STD_ID" VARCHAR2(5),
"STD_NAME" VARCHAR2(10),
"STD_FNAME" VARCHAR2(10),
"STD_DOB" DATE,
"STD_EMAIL" VARCHAR2(7),
"STD_ADRESS" VARCHAR2(10),
"STD_PHONE" NUMBER(11)
)
```

Results Explain Describe Saved SQL History

Table created.

0.02 seconds

Figure 10: Table Creation Confirmation

DESCRIBE "TUF_STUDENTS";

Results Explain **Describe** Saved SQL History

Object Type **TABLE** Object **TUF_STUDENTS**

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
TUF_STUDENTS	STD_ID	Varchar2	5	-	-	-	✓	-	-
	STD_NAME	Varchar2	10	-	-	-	✓	-	-
	STD_FNAME	Varchar2	10	-	-	-	✓	-	-
	STD_DOB	Date	7	-	-	-	✓	-	-
	STD_EMAIL	Varchar2	7	-	-	-	✓	-	-
	STD_ADRESS	Varchar2	10	-	-	-	✓	-	-
	STD_PHONE	Number	-	11	0	-	✓	-	-
1 - 7									

Figure 11: Table Structure of TUF_STUDENTS

1.2. Task No 2:

```
CREATE TABLE "GPTS" (
  "CODING_GPT" VARCHAR2(10),
  "MATH_GPT" VARCHAR2(10),
  "NUMBER_OF_USERS" NUMBER(10),
  "LAUNCH_DATE" DATE,
  "RATING" FLOAT
);
```

Results Explain Describe Saved SQL History

Table created.

0.01 seconds

Figure 12: Table Creation Confirmation

Task 2:

Making Changes to Table using ALTER Command

```
DESCRIBE "TUF_STUDENTS";
```

Results Explain Describe Saved SQL History

Object Type **TABLE** Object **GPTS**

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
<u>GPTS</u>	<u>CODING_GPT</u>	Varchar2	10	-	-	-	✓	-	-
	<u>MATH_GPT</u>	Varchar2	10	-	-	-	✓	-	-
	<u>NUMBER_OF_USERS</u>	Number	-	10	0	-	✓	-	-
	<u>LAUNCH_DATE</u>	Date	7	-	-	-	✓	-	-
	<u>RATING</u>	Float	22	126	-	-	✓	-	-
1 - 5									

Figure 13: Table Structure of GPTS

2.1 Alter Table

```
ALTER TABLE TUF_STUDENTS
```

```
ADD STD_PERMANENT_ADRESS VARCHAR2(50);
```

Results Explain Describe Saved SQL History

Table altered.

0.00 seconds

Figure 14: Table Alteration Confirmation

Table Structure After Adding One Column:

```
DESCRIBE "TUF_STUDENTS";
```

Results Explain Describe Saved SQL History

Object Type **TABLE** Object **TUF_STUDENTS**

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
TUF_STUDENTS	STD_ID	Varchar2	5	-	-	-	✓	-	-
	STD_NAME	Varchar2	10	-	-	-	✓	-	-
	STD_FNAME	Varchar2	10	-	-	-	✓	-	-
	STD_DOB	Date	7	-	-	-	✓	-	-
	STD_EMAIL	Varchar2	7	-	-	-	✓	-	-
	STD_ADDRESS	Varchar2	10	-	-	-	✓	-	-
	STD_PHONE	Number	-	11	0	-	✓	-	-
	STD_PERMANENT_ADRESS	Varchar2	50	-	-	-	✓	-	-
									1 - 8

Figure 15: Updated Table Structure of TUF_STUDENTS

2.1.1 Rename Column:

Query:

```
ALTER TABLE TUF_STUDENTS
```

```
RENAME COLUMN STD_PERMANENT_ADRESS TO RELIGION;
```

Results Explain Describe Saved SQL History

Table altered.

0.02 seconds

Figure 16: Table Alter Confirmation

2.2 Modify Column

Query:

```
ALTER TABLE TUF_STUDENTS
```

```
MODIFY RELIGION VARCHAR(10);
```

Results Explain Describe Saved SQL History

Table altered.

0.01 seconds

Figure 17: Table Alter Confirmation

2.3 Using DROP Command for Dropping Column

2.3.1 DROP QUERY

ALTER TABLE TUF_STUDENTS

DROP COLUMN GENDER;

Results Explain Describe Saved SQL History

Table altered.

0.03 seconds

Figure 18: Table Alter Confirmation

Results Explain Describe Saved SQL History

Object Type **TABLE** Object **TUF_STUDENTS**

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
TUF_STUDENTS	STD_ID	Varchar2	5	-	-	-	✓	-	-
	STD_NAME	Varchar2	10	-	-	-	✓	-	-
	STD_FNAME	Varchar2	10	-	-	-	✓	-	-
	STD_DOB	Date	7	-	-	-	✓	-	-
	STD_EMAIL	Varchar2	7	-	-	-	✓	-	-
	STD_ADRESS	Varchar2	10	-	-	-	✓	-	-
	STD_PHONE	Number	-	11	0	-	✓	-	-
	RELIGION	Varchar2	10	-	-	-	✓	-	-

1 - 8

Figure 19: Table Structure of TUF_STUDENTS

2.4 Rename Table Using RENAME Command

RENAME TABLE TUF_STUDENTS TO BSAI

DESCRIBE "BSAI";

Results Explain Describe Saved SQL History

Object Type **TABLE** Object **BSAI**

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
BSAI	STD_ID	Varchar2	5	-	-	-	✓	-	-
	STD_NAME	Varchar2	10	-	-	-	✓	-	-
	STD_FNAME	Varchar2	10	-	-	-	✓	-	-
	STD_DOB	Date	7	-	-	-	✓	-	-
	STD_EMAIL	Varchar2	7	-	-	-	✓	-	-
	STD_ADRESS	Varchar2	10	-	-	-	✓	-	-
	STD_PHONE	Number	-	11	0	-	✓	-	-
	RELIGION	Varchar2	10	-	-	-	✓	-	-

1 - 8

Figure 20: Table Structure of BSAI

Task 3:

```
CREATE TABLE "LOGIN_PAGE" (
  "ID" VARCHAR(10),
  "USERNAME" VARCHAR2(10),
  "PASSWORD" NUMBER(10)
);
```

Results Explain Describe Saved SQL History

Table created.

0.00 seconds

Figure 21: Table Creation Confirmation

DESCRIBE "LOGIN_PAGE";

Results Explain Describe Saved SQL History

Object Type **TABLE** Object **LOGIN_PAGE**

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
LOGIN_PAGE	ID	Varchar2	10	-	-	-	✓	-	-
	USERNAME	Varchar2	10	-	-	-	✓	-	-
	PASSWORD	Number	-	10	0	-	✓	-	-

1 - 3

Figure 22: Table Structure of LOGIN_PAGE

```
DROP TABLE "LOGIN_PAGE";
```

Results Explain Describe Saved SQL History

Table dropped.

0.20 seconds

Figure 23: Table Deleted

```
DESCRIBE "LOGIN_PAGE";
```

Results Explain Describe Saved SQL History

Object to be described could not be found.

Figure 24: There is no LOGIN_PAGE Table in Database

LAB 3: INSERT, UPDATE, DELETE TABLE DATA USING DML

Task 1:

Inserting Values in the Table using INSERT Command

1. Create “STUDENT_RESULT” Table

```
CREATE TABLE "STUDENT_RESULT"
( "STUDENT_ID" NUMBER(3,0) NOT NULL ENABLE,
  "STUDENT_NAME" VARCHAR2(20),
  "SUB_MARKS" NUMBER(5,0),
  "SUB_GPA" NUMBER NOT NULL ENABLE,
  "SUB_GRADE" CHAR(1),
  "SUB_NAME" VARCHAR2(20)
)
```

Results Explain Describe Saved SQL History

Table created.

0.06 seconds

Figure 25: Table Creation Confirmation

1.2 Adding STUDENT_SEMESTER Column

```
ALTER TABLE STUDENT_RESULT
ADD STUDENT_SEMESTER VARCHAR2(3)
```

Results Explain Describe Saved SQL History

Table altered.

0.03 seconds

Figure 26: Table Updated

1.3 Renaming Column

```
ALTER TABLE STUDENT_RESULT
```

```
RENAME COLUMN SUB_CGPA TO SUB_GPA
```

Results Explain Describe Saved SQL History

ORA-00957: duplicate column name

0.00 seconds

Figure 27: Error

1.4 Insert Query

```
INSERT INTO STD_RESULT(STUDENT_ID, STUDENT_NAME, SUB_MARKS, SUB_GPA,
SUB_GRADE, SUB_NAME)
```

```
VALUES (01, 'ABC', 65, 3.8, 'A', 'DB')
```

Results Explain Describe Saved SQL History

1 row(s) inserted.

0.02 seconds

Figure 28: New Row Added

1.5 Select Query

```
SELECT * FROM STD_RESULT;
```

Results Explain Describe Saved SQL History

STUDENT_ID	STUDENT_NAME	SUB_MARKS	SUB_GPA	SUB_GRADE	SUB_NAME
1	ABC	65	3.8	A	DB

1 rows returned in 0.00 seconds

[CSV Export](#)

Figure 29: Data in STD_RESULT

1.6 Inserting another Row in existing Data

```
INSERT INTO STD_RESULT(STUDENT_ID, STUDENT_NAME, SUB_MARKS, SUB_GPA,
SUB_GRADE, SUB_NAME)
```

```
VALUES (003, 'JKL', 59, 3.9, 'B', 'DB');
```

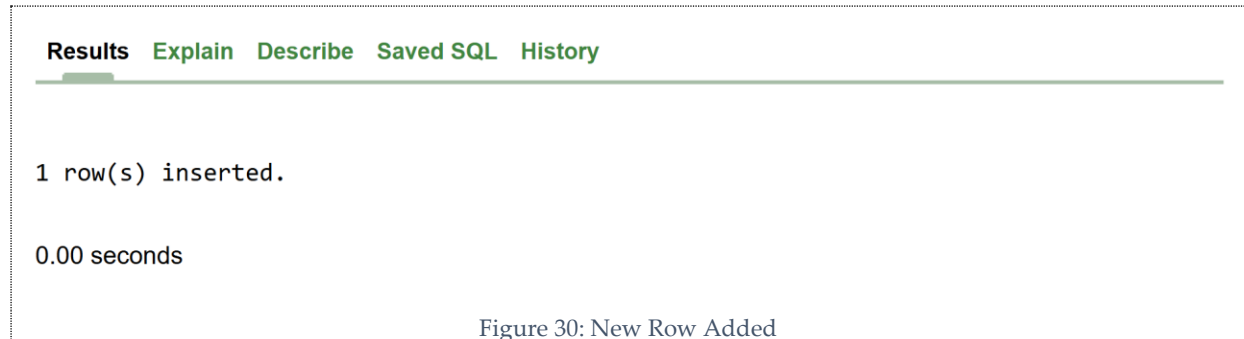


Figure 30: New Row Added

Task 2:

Inserting Multiple Values in the Table using INSERT ALL Command

2.1 Single statement:

```
INSERT INTO TUF_STUDENTS
```

```
VALUES ('2023-BS-AI-017', 'HAMNA', TO_DATE('01-08-2023', 'DD-MM-
YYYY'), '+923669998996', '33100-5555092-0');
```

2.2 Dual statement:

```
INSERT ALL
```

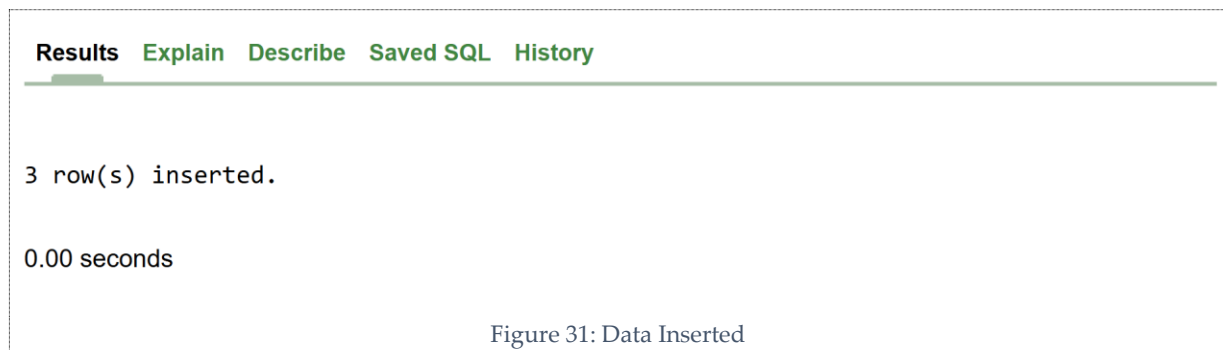
```
INTO TUF_STUDENTS
```

```
VALUES ('2023-BS-AI-020', 'MISHAL', TO_DATE('22-08-2023', 'DD-MM-
YYYY'), '+923998865409', '33100-59875347-0')
```

```
INTO TUF_STUDENTS
```

```
VALUES ('2023-BS-AI-021', 'YUMNA', TO_DATE('24-08-2023', 'DD-MM-
YYYY'), '+923998654408', '33100-75314579-0')
```

```
SELECT * FROM dual;
```



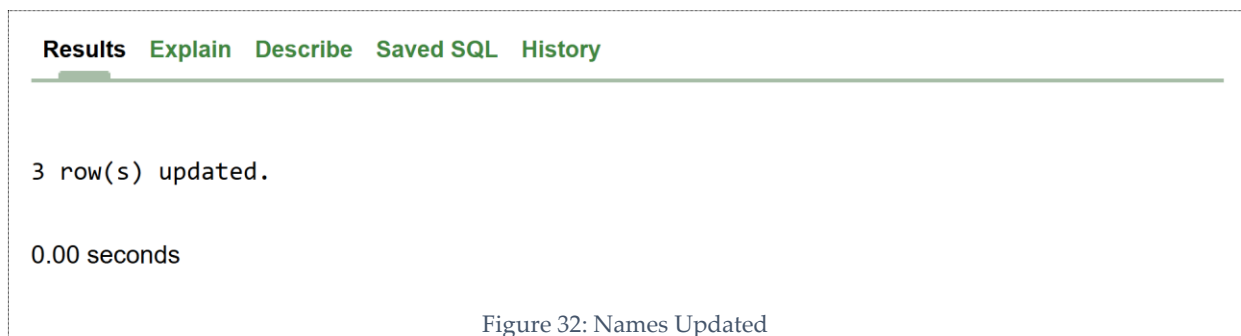
Task 3:

Update Table Using UPDATE Command

3.1 Updating All Values of TUF_STUDENTS

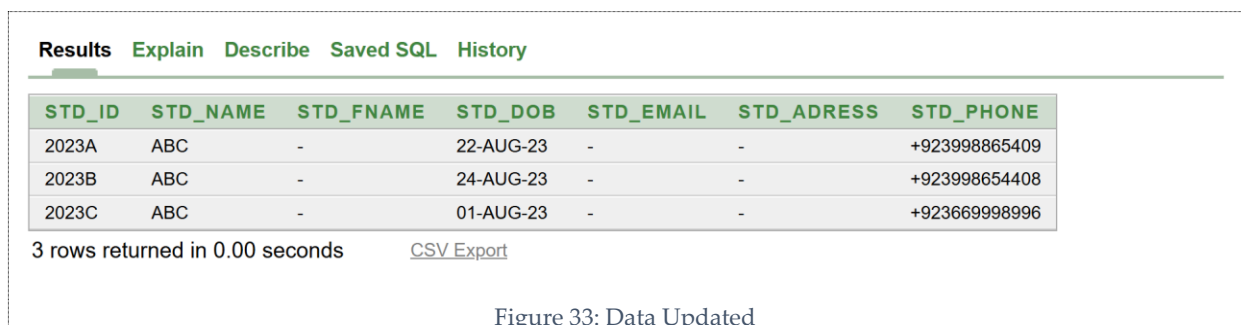
```
UPDATE TUF_STUDENTS
```

```
SET NAME = 'ABC'
```



3.1.1 Updated Structure of Table

```
SELECT * FROM TUF_STUDENTS;
```

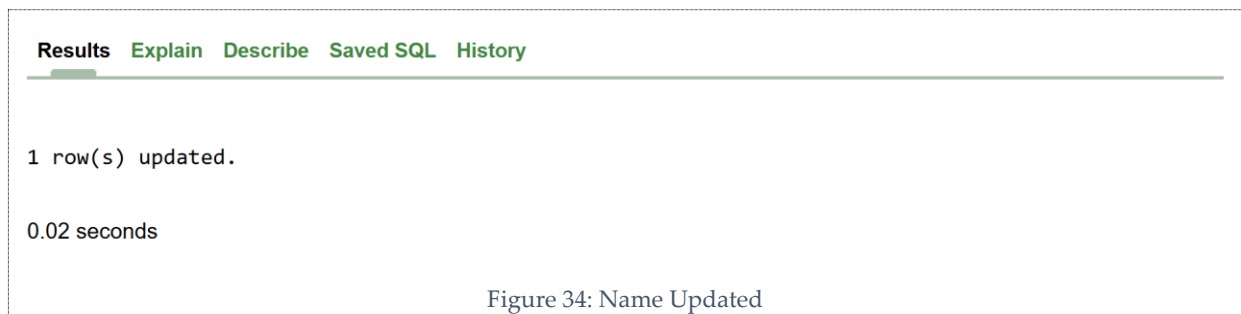


3.1.2 Updating Specific Values Using WHERE Command

```
UPDATE TUF_STUDENTS
```

```
SET NAME = 'XYZ'
```

```
WHERE REGISTRATION = '2023-BS-AI-017';
```



3.1.3 Delete Table TUF_STUDENTS

```
DELETE TUF_STUDENTS
```

LAB 4: THE PRACTICE OF DDL AND DML QUERIES WITH DIFFERENT CASE SCENARIOS

SCENARIO # 1

MANAGING EMPLOYEE RECORDS IN A COMPANY

Task 1

Creating a table for employees

```
CREATE TABLE EMPLOYEE_INFO (
    EMPLOYEE_ID VARCHAR2(9),
    EMPLOYEE_NAME VARCHAR2(20),
    EMPLOYEE_AGE NUMBER(2),
    EMPLOYEE_DEPT VARCHAR2(10)
);
```

Results

Explain

Describe

Saved SQL

History

Object Type

TABLE

Object

EMPLOYEE_INFO

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
EMPLOYEE_INFO	EMPLOYEE_ID	Varchar2	9	-	-	-	✓	-	-
	EMPLOYEE_NAME	Varchar2	20	-	-	-	✓	-	-
	EMPLOYEE_AGE	Number	-	2	0	-	✓	-	-
	EMPLOYEE_DEPT	Varchar2	10	-	-	-	✓	-	-
									1 - 4

Figure 35: Structure of "Employee_INFO"

Figure 35: Structure of "Employee_INFO"

1.2 Adding Column of Salary

```
ALTER TABLE EMPLOYEE_INFO
```

```
ADD SALARY NUMBER
```

Results	Explain	Describe	Saved SQL	History
Table altered.				
0.02 seconds				

Figure 36: "Salary" Added in "Employee_INFO"

1.3 Rename Table from “EMPLOYEE_INFO” to “EMPLOYEE_DETAILS”

```
RENAME "EMPLOYEE_INFO" TO "EMPLOYEE_DETAILS"
```

Results Explain Describe Saved SQL History

Statement processed.

0.00 seconds

Figure 37: Table Renamed

1.4 Inserting Data in Table

```
INSERT ALL
```

```
    INTO EMPLOYEE_DETAILS (EMPLOYEE_ID, EMPLOYEE_NAME, EMPLOYEE_AGE,
EMPLOYEE_DEPT, SALARY) VALUES ('E001', 'Aisha Khan', 27, 'HR', 55000)
```

```
    INTO EMPLOYEE_DETAILS (EMPLOYEE_ID, EMPLOYEE_NAME, EMPLOYEE_AGE,
EMPLOYEE_DEPT, SALARY) VALUES ('E002', 'Hina Ahmed', 32, 'Finance',
72000)
```

```
    INTO EMPLOYEE_DETAILS (EMPLOYEE_ID, EMPLOYEE_NAME, EMPLOYEE_AGE,
EMPLOYEE_DEPT, SALARY) VALUES ('E003', 'Maira Riaz', 40, 'IT', 85000)
```

```
    INTO EMPLOYEE_DETAILS (EMPLOYEE_ID, EMPLOYEE_NAME, EMPLOYEE_AGE,
EMPLOYEE_DEPT, SALARY) VALUES ('E004', 'Sadia Noor', 29, 'Marketing',
60000)
```

```
    INTO EMPLOYEE_DETAILS (EMPLOYEE_ID, EMPLOYEE_NAME, EMPLOYEE_AGE,
EMPLOYEE_DEPT, SALARY) VALUES ('E005', 'Zoya Malik', 26, 'Sales',
50000)
```

```
SELECT * FROM dual;
```

Results Explain Describe Saved SQL History

EMPLOYEE_ID	EMPLOYEE_NAME	EMPLOYEE_AGE	EMPLOYEE_DEPT	SALARY
E001	Aisha Khan	27	HR	55000
E002	Hina Ahmed	32	Finance	72000
E003	Maira Riaz	40	IT	85000
E004	Sadia Noor	29	Marketing	60000
E005	Zoya Malik	26	Sales	50000

5 rows returned in 0.00 seconds [CSV Export](#)

Figure 38: Data Entered

1.5 Updated IT Employee to HR

```
UPDATE EMPLOYEE_DETAILS
```

```
SET EMPLOYEE_DEPT = 'HR'
```

```
WHERE EMPLOYEE_DEPT = 'IT';
```

Results	Explain	Describe	Saved SQL	History																														
<table border="1"> <thead> <tr> <th>EMPLOYEE_ID</th><th>EMPLOYEE_NAME</th><th>EMPLOYEE_AGE</th><th>EMPLOYEE_DEPT</th><th>SALARY</th></tr> </thead> <tbody> <tr> <td>E001</td><td>Aisha Khan</td><td>27</td><td>HR</td><td>55000</td></tr> <tr> <td>E002</td><td>Hina Ahmed</td><td>32</td><td>Finance</td><td>72000</td></tr> <tr> <td>E003</td><td>Maira Riaz</td><td>40</td><td>HR</td><td>85000</td></tr> <tr> <td>E004</td><td>Sadia Noor</td><td>29</td><td>Marketing</td><td>60000</td></tr> <tr> <td>E005</td><td>Zoya Malik</td><td>26</td><td>Sales</td><td>50000</td></tr> </tbody> </table>					EMPLOYEE_ID	EMPLOYEE_NAME	EMPLOYEE_AGE	EMPLOYEE_DEPT	SALARY	E001	Aisha Khan	27	HR	55000	E002	Hina Ahmed	32	Finance	72000	E003	Maira Riaz	40	HR	85000	E004	Sadia Noor	29	Marketing	60000	E005	Zoya Malik	26	Sales	50000
EMPLOYEE_ID	EMPLOYEE_NAME	EMPLOYEE_AGE	EMPLOYEE_DEPT	SALARY																														
E001	Aisha Khan	27	HR	55000																														
E002	Hina Ahmed	32	Finance	72000																														
E003	Maira Riaz	40	HR	85000																														
E004	Sadia Noor	29	Marketing	60000																														
E005	Zoya Malik	26	Sales	50000																														
5 rows returned in 0.00 seconds CSV Export																																		

Figure 39: Department Updated

1.6 Deleting Data of Employee

```
DELETE FROM EMPLOYEE_DETAILS
```

Results	Explain	Describe	Saved SQL	History
5 row(s) deleted. 0.00 seconds				

Figure 40: Data Deleted from Table

SCENARIO # 2

MANAGING A LIBRARY SYSTEM

Task 2

2.1 Creating “Book” Table

```
CREATE TABLE BOOKS (
```

```
    ID NUMBER PRIMARY KEY,
```

```
    BOOK_TITLE VARCHAR2(25),
```

```
    WRITER VARCHAR2(25),
```

```
    YEAR_PUBLISHED DATE
```

```
);
```

Results

Explain

Describe

Saved SQL

History

Object Type

TABLE

Object

BOOKS

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
BOOKS	ID	Number	-	-	-	1	-	-	-
	BOOK_TITLE	Varchar2	25	-	-	-	✓	-	-
	WRITER	Varchar2	25	-	-	-	✓	-	-
	YEAR_PUBLISHED	Date	7	-	-	-	✓	-	-
									1 - 4

Figure 41: Table "Book" Created

2.2 Addinng a Column “GENRE”

```
ALTER TABLE BOOKS
```

```
ADD GENRE VARCHAR2(30);
```

Results

Explain

Describe

Saved SQL

History

Object Type

TABLE

Object

BOOKS

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
BOOKS	ID	Number	-	-	-	1	-	-	-
	BOOK_TITLE	Varchar2	25	-	-	-	✓	-	-
	WRITER	Varchar2	25	-	-	-	✓	-	-
	YEAR_PUBLISHED	Date	7	-	-	-	✓	-	-
	GENRE	Varchar2	30	-	-	-	✓	-	-
1 - 5									

Figure 42: New column Added

2.3 Insert Data in “BOOKS” Table

```
INSERT ALL
```

```
  INTO books (ID, BOOK_TITLE, WRITER, YEAR_PUBLISHED, GENRE) VALUES
(1, 'Riyad us-Saliheen', 'Imam Nawawi', TO_DATE('1250-01-01', 'YYYY-MM-DD'), 'Hadith')
```

```
  INTO books (ID, BOOK_TITLE, WRITER, YEAR_PUBLISHED, GENRE) VALUES
(2, 'Sahih al-Bukhari', 'Imam Bukhari', TO_DATE('0846-01-01', 'YYYY-MM-DD'), 'Hadith')
```

```
  INTO books (ID, BOOK_TITLE, WRITER, YEAR_PUBLISHED, GENRE) VALUES
(3, 'Sahih Muslim', 'Imam Muslim', TO_DATE('0875-01-01', 'YYYY-MM-DD'), 'Hadith')
```

```

    INTO books (ID, BOOK_TITLE, WRITER, YEAR_PUBLISHED, GENRE) VALUES
    (4, 'Fiqh-us-Sunnah', 'Sayyid Sabiq', TO_DATE('1960-01-01', 'YYYY-MM-DD'), 'Fiqh')

```

```

    INTO books (ID, BOOK_TITLE, WRITER, YEAR_PUBLISHED, GENRE) VALUES
    (5, 'Tafsir al-Jalalayn', 'Jalaluddin Mahalli', TO_DATE('1505-01-01', 'YYYY-MM-DD'), 'Tafsir')

```

```

    INTO books (ID, BOOK_TITLE, WRITER, YEAR_PUBLISHED, GENRE) VALUES
    (6, 'Ihya Ulum al-Din', 'Imam al-Ghazali', TO_DATE('1100-01-01', 'YYYY-MM-DD'), 'Spirituality')

```

```

    INTO books (ID, BOOK_TITLE, WRITER, YEAR_PUBLISHED, GENRE) VALUES
    (7, 'Al-Muwatta', 'Imam Malik', TO_DATE('0795-01-01', 'YYYY-MM-DD'), 'Hadith/Fiqh')

```

```

    INTO books (ID, BOOK_TITLE, WRITER, YEAR_PUBLISHED, GENRE) VALUES
    (8, 'The Book of Knowledge', 'Imam al-Ghazali', TO_DATE('1100-01-01', 'YYYY-MM-DD'), 'Spirituality')

```

```

    INTO books (ID, BOOK_TITLE, WRITER, YEAR_PUBLISHED, GENRE) VALUES
    (9, 'Kitab al-Tawheed', 'Ibn Abd al-Wahhab', TO_DATE('1744-01-01', 'YYYY-MM-DD'), 'Aqidah')

```

```

    INTO books (ID, BOOK_TITLE, WRITER, YEAR_PUBLISHED, GENRE) VALUES
    (10, 'Tafsir Ibn Kathir', 'Ibn Kathir', TO_DATE('1373-01-01', 'YYYY-MM-DD'), 'Tafsir')

```

```

SELECT * FROM dual;

```

Results	Explain	Describe	Saved SQL	History
ID	BOOK_TITLE	WRITER	YEAR_PUBLISHED	GENRE
1	Riyad us-Saliheen	Imam Nawawi	01-JAN-50	Hadith
2	Sahih al-Bukhari	Imam Bukhari	01-JAN-46	Hadith
3	Sahih Muslim	Imam Muslim	01-JAN-75	Hadith
4	Fiqh-us-Sunnah	Sayyid Sabiq	01-JAN-60	Fiqh
5	Tafsir al-Jalalayn	Jalaluddin Mahalli	01-JAN-05	Tafsir
6	Ihya Ulum al-Din	Imam al-Ghazali	01-JAN-00	Spirituality
7	Al-Muwatta	Imam Malik	01-JAN-95	Hadith/Fiqh
8	The Book of Knowledge	Imam al-Ghazali	01-JAN-00	Spirituality
9	Kitab al-Tawheed	Ibn Abd al-Wahhab	01-JAN-44	Aqidah
10	Tafsir Ibn Kathir	Ibn Kathir	01-JAN-73	Tafsir

10 rows returned in 0.01 seconds [CSV Export](#)

Figure 43: Data Inserted in Table "Books"

2.4 Update Books' Genre to "Islamic Book"

```
UPDATE BOOKS SET GENRE='Islamic Book'
```

Results	Explain	Describe	Saved SQL	History
ID	BOOK_TITLE	WRITER	YEAR_PUBLISHED	GENRE
1	Riyad us-Saliheen	Imam Nawawi	01-JAN-50	Islamic Book
2	Sahih al-Bukhari	Imam Bukhari	01-JAN-46	Islamic Book
3	Sahih Muslim	Imam Muslim	01-JAN-75	Islamic Book
4	Fiqh-us-Sunnah	Sayyid Sabiq	01-JAN-60	Islamic Book
5	Tafsir al-Jalalayn	Jalaluddin Mahalli	01-JAN-05	Islamic Book
6	Ihya Ulum al-Din	Imam al-Ghazali	01-JAN-00	Islamic Book
7	Al-Muwatta	Imam Malik	01-JAN-95	Islamic Book
8	The Book of Knowledge	Imam al-Ghazali	01-JAN-00	Islamic Book
9	Kitab al-Tawheed	Ibn Abd al-Wahhab	01-JAN-44	Islamic Book
10	Tafsir Ibn Kathir	Ibn Kathir	01-JAN-73	Islamic Book

10 rows returned in 0.00 seconds [CSV Export](#)

Figure 44: Genre of All Books Updated

2.5 Delete Data from "BOOKS"

```
DELETE FROM BOOKS
```

Results	Explain	Describe	Saved SQL	History
10 row(s) deleted.				
0.00 seconds				

Figure 45: Data Deleted

SCENARIO # 3

MANAGING AN ONLINE STORE'S PRODUCT INVENTORY

Task 3

3.1 Creating a table for products

```
CREATE TABLE Products (
    ProductID NUMBER PRIMARY KEY,
    Name VARCHAR2(20) NOT NULL,
    UnitPrice NUMBER UNIQUE,
    InStock CHAR(1)
);
```

Results

Explain

Describe

Saved SQL

History

Object Type

TABLE

Object

PRODUCTS

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
PRODUCTS	PRODUCTID	Number	-	-	-	1	-	-	-
	NAME	Varchar2	20	-	-	-	-	-	-
	UNITPRICE	Number	-	-	-	-	✓	-	-
	INSTOCK	Char	1	-	-	-	✓	-	-
									1 - 4

Figure 46: Table "Products" Created

Figure 46: Table "Products" Created

3.2 Adding new column Name “SUPPLIER”

```
ALTER TABLE PRODUCTS
ADD SUPPLIER VARCHAR2(20)
```

Results	Explain	Describe	Saved SQL	History
Table altered.				
0.01 seconds				

Figure 47: Column "Supplier" Added

3.3 Rename Table from “Products” to “StoreProducts”

RENAME "PRODUCTS" TO "STOREPRODUCTS"

Results
Explain
Describe
Saved SQL
History

Statement processed.

0.00 seconds

Figure 48: Table Name Changed

3.4 Inseting Values in “StoreProducts”

INSERT ALL

INTO STOREPRODUCTS (PRODUCTID, NAME, UNITPRICE, INSTOCK, SUPPLIER)
VALUES (1, 'Keyboard', 25.50, 'Y', 'Aaliyah')

INTO STOREPRODUCTS (PRODUCTID, NAME, UNITPRICE, INSTOCK, SUPPLIER)
VALUES (2, 'Mouse', 15.00, 'N', 'Fatimah')

INTO STOREPRODUCTS (PRODUCTID, NAME, UNITPRICE, INSTOCK, SUPPLIER)
VALUES (3, 'Monitor', 130.75, 'Y', 'Zainab')

INTO STOREPRODUCTS (PRODUCTID, NAME, UNITPRICE, INSTOCK, SUPPLIER)
VALUES (4, 'Printer', 112.20, 'N', 'Maryam')

INTO STOREPRODUCTS (PRODUCTID, NAME, UNITPRICE, INSTOCK, SUPPLIER)
VALUES (5, 'Webcam', 45.00, 'Y', 'Khadijah')

SELECT * FROM DUAL;

Results
Explain
Describe
Saved SQL
History

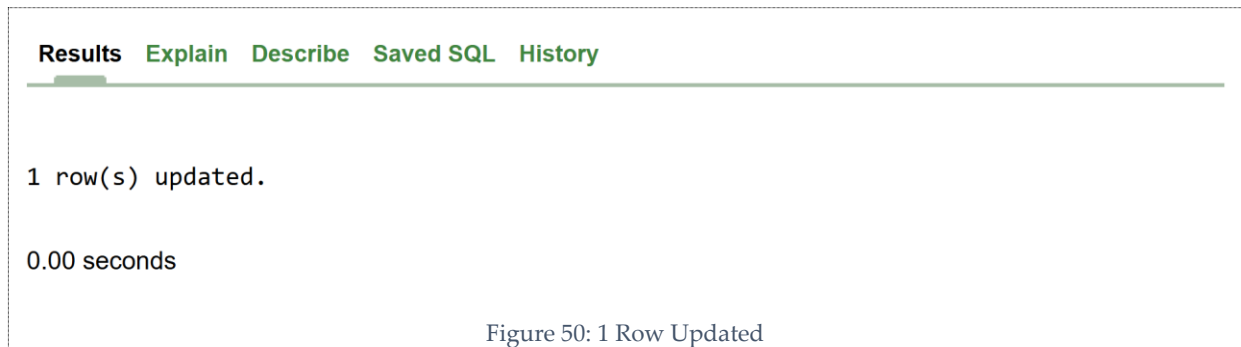
PRODUCTID	NAME	UNITPRICE	INSTOCK	SUPPLIER
1	Keyboard	25.5	Y	Aaliyah
2	Mouse	15	N	Fatimah
3	Monitor	130.75	Y	Zainab
4	Printer	112.2	N	Maryam
5	Webcam	45	Y	Khadijah

5 rows returned in 0.00 seconds [CSV Export](#)

Figure 49: Data Entered

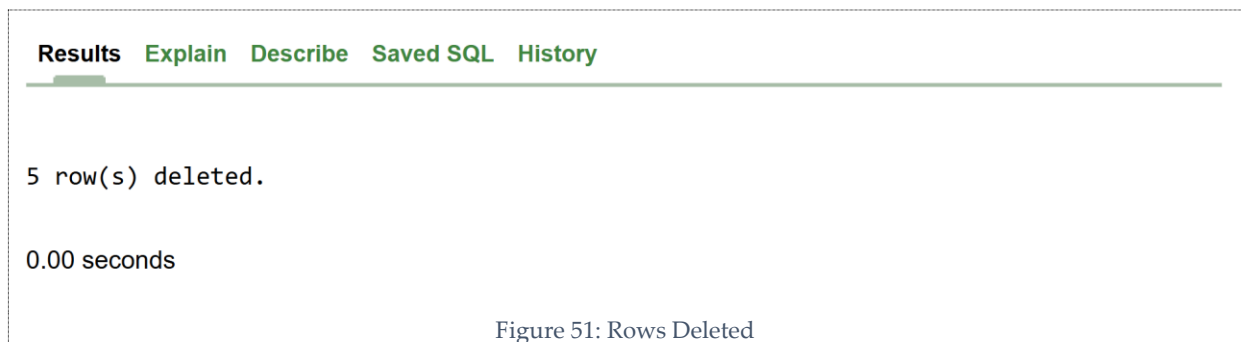
3.5 Update StoreProduct

```
UPDATE STOREPRODUCTS SET UNITPRICE = 100 WHERE NAME = 'Keyboard';
```



3.6 Delete “StoreProducts” Record

```
DELETE FROM STOREPRODUCTS
```



SCENARIO # 4

MANAGING A HOSPITAL'S PATIENT RECORDS

Task 4:

4.1 Creating a table for patients

```
CREATE TABLE PATIENT (
    PATIENT_ID INT PRIMARY KEY,
    NAME VARCHAR(100) NOT NULL,
    AGE INT,
    DISEASE VARCHAR(100) UNIQUE
);
```

DESCRIBE PATIENT

Results

Explain

Describe

Saved SQL

History

Object Type

TABLE

Object

PATIENT

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
PATIENT	PATIENT_ID	Number	-	-	0	1	-	-	-
	NAME	Varchar2	100	-	-	-	-	-	-
	AGE	Number	-	-	0	-	✓	-	-
	DISEASE	Varchar2	100	-	-	-	✓	-	-
									1 - 4

Figure 52: Table Structure of PATIENT

Figure 52: Table Structure of PATIENT

4.2 Add New Column Doctor_Name

ALTER TABLE PATIENT

ADD DOCTOR_NAME VARCHAR(100);

Results	Explain	Describe	Saved SQL	History
Table altered.				
0.05 seconds				

Figure 53: Table Updated

4.3 Rename Table Name

ALTER TABLE PATIENT

RENAME TO PATIENT_RECORDS;

Results	Explain	Describe	Saved SQL	History
Table altered.				
0.02 seconds				

Figure 54: Table Altered Confirmation

DESCRIBE PATIENT_RECORDS

Results Explain Describe Saved SQL History									
Object Type TABLE Object PATIENT_RECORDS									
Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
PATIENT_RECORDS	PATIENT_ID	Number	-	-	0	1	-	-	-
	NAME	Varchar2	100	-	-	-	-	-	-
	AGE	Number	-	-	0	-	✓	-	-
	DISEASE	Varchar2	100	-	-	-	✓	-	-
	DOCTOR_NAME	Varchar2	100	-	-	-	✓	-	-
									1 - 5

Figure 55: Structure of PATIENT_RECORDS

4.4 Insert Data in Table

INSERT ALL

```

INTO PATIENT_RECORDS VALUES (1, 'AYESHA', 45, 'Diabetes', 'DR.KAMRAN')
INTO PATIENT_RECORDS VALUES (2, 'SANA', 30, 'Hypertension', 'DR.AWAN')
INTO PATIENT_RECORDS VALUES (3, 'NIDA', 55, 'Asthma', 'DR.AFZAAL')
INTO PATIENT_RECORDS VALUES (4, 'HINA', 40, 'Arthritis', 'DR.KHALID')
INTO PATIENT_RECORDS VALUES (5, 'MARIAM', 60, 'Heart Disease',
'DR.ARIF')

SELECT * FROM DUAL;

```

Results Explain Describe Saved SQL History				
PATIENT_ID	NAME	AGE	DISEASE	DOCTOR_NAME
1	AYESHA	45	Diabetes	DR.KAMRAN
2	SANA	30	Hypertension	DR.AWAN
3	NIDA	55	Asthma	DR.AFZAAL
4	HINA	40	Arthritis	DR.KHALID
5	MARIAM	60	Heart Disease	DR.ARIF

5 rows returned in 0.01 seconds [CSV Export](#)

Figure 56: Data Inserted

4.5 Delete a Patient Record

DELETE FROM PATIENT_RECORDS WHERE PATIENT_ID = 1

Results

Explain

Describe

Saved SQL

History

Object Type

TABLE

Object

PATIENT_RECORDS

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
PATIENT_RECORDS	PATIENT_ID	Number	-	-	0	1	-	-	-
	NAME	Varchar2	100	-	-	-	-	-	-
	AGE	Number	-	-	0	-	✓	-	-
	DISEASE	Varchar2	100	-	-	-	✓	-	-
	DOCTOR_NAME	Varchar2	100	-	-	-	✓	-	-
									1 - 5

Figure 57: Deleted Record

Figure 57: Deleted Record

LAB 5: WHERE AND ORDER BY CLAUSE

Task # 1

SELECT *FROM employees

Results Explain Describe Saved SQL History

EMPLOYEE_ID	FIRST_NAME	LAST_NAME	EMAIL	PHONE_NUMBER	HIRE_DATE	JOB_ID	SALARY	COMMISSION_PCT	MANAGER_ID	DEPARTMENT_ID
100	Steven	King	SKING	515.123.4567	17-JUN-87	AD_PRES	24000	-	-	90
101	Neena	Kochhar	NKOCHHAR	515.123.4568	21-SEP-89	AD_VP	17000	-	100	90
102	Lex	De Haan	LDEHAAN	515.123.4569	13-JAN-93	AD_VP	17000	-	100	90
103	Alexander	Hunold	AHUNOLD	590.423.4567	03-JAN-90	IT_PROG	9000	-	102	60
104	Bruce	Ernst	BERNST	590.423.4568	21-MAY-91	IT_PROG	6000	-	103	60
105	David	Austin	DAUSTIN	590.423.4569	25-JUN-97	IT_PROG	4800	-	103	60
106	Valli	Pataballa	VPATABAL	590.423.4560	05-FEB-98	IT_PROG	4800	-	103	60
107	Diana	Lorentz	DLORENTZ	590.423.5567	07-FEB-99	IT_PROG	4200	-	103	60
108	Nancy	Greenberg	NGREENBE	515.124.4569	17-AUG-94	FI_MGR	12000	-	101	100
109	Daniel	Faviet	DFAVIET	515.124.4169	16-AUG-94	FI_ACCOUNT	9000	-	108	100

More than 10 rows available. Increase rows selector to view more rows.

10 rows returned in 0.00 seconds [CSV Export](#)

Figure 58: Data in table "employees"

Query:

For specific number of columns:

SELECT EMPLOYEE_ID, LAST_NAME FROM employees

Results Explain Describe Saved SQL History

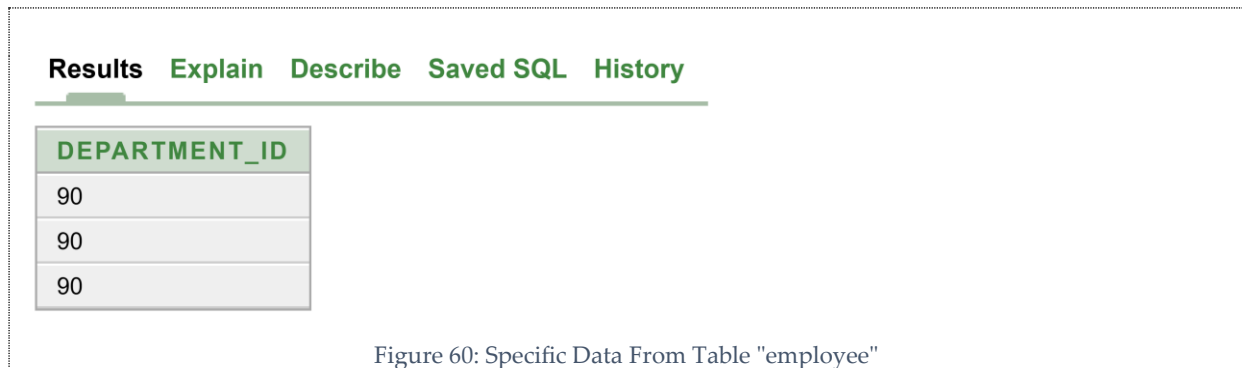
EMPLOYEE_ID	LAST_NAME
100	King
101	Kochhar
102	De Haan
103	Hunold
104	Ernst
105	Austin
106	Pataballa
107	Lorentz
108	Greenberg
109	Faviet

More than 10 rows available. Increase rows selector to view more rows.

Figure 59: Show Specific Data

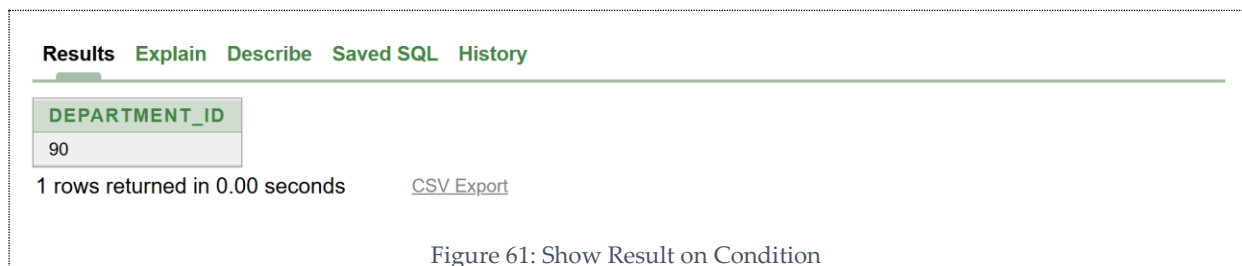
Query:

```
SELECT DEPARTMENT_ID FROM employees
WHERE DEPARTMENT_ID = 90;
```



```
SELECT DEPARTMENT_ID FROM employees
WHERE LAST_NAME=grant;
```

```
SELECT DEPARTMENT_ID FROM employees
WHERE SALARY>20000;
```



Where Clause Can't Come Before Table

```

SELECT EMPLOYEE_ID, FIRST_NAME, DEPARTMENT_ID
FROM employees
WHERE JOB_ID = 'IT_PROG';

```

Results

Explain

Describe

Saved SQL

History

EMPLOYEE_ID	FIRST_NAME	DEPARTMENT_ID
103	Alexander	60
104	Bruce	60
105	David	60
106	Valli	60
107	Diana	60

5 rows returned in 0.01 seconds

[CSV Export](#)

Figure 62: Show Specific Data from Table "employees"

```

SELECT LAST_NAME, SALARY
FROM employees
WHERE SALARY BETWEEN 2500 AND 3500;

```

Results

Explain

Describe

Saved SQL

History

LAST_NAME	SALARY
Khoo	3100
Baida	2900
Tobias	2800
Himuro	2600
Colmenares	2500
Nayer	3200
Mikkilineni	2700
Bissot	3300
Atkinson	2800
Marlow	2500
More than 10 rows available. Increase rows selector to view more rows.	

Figure 63: Show Specific Data from Table "employees"

```
SELECT LAST_NAME, SALARY
FROM employees
WHERE manager_id IN (100,102,103);
```

Results

Explain

Describe

Saved SQL

History

LAST_NAME	SALARY
Kochhar	17000
De Haan	17000
Raphaely	11000
Weiss	8000
Fripp	8200
Kaufling	7900
Vollman	6500
Mourgos	5800
Russell	14000
Partners	13500
More than 10 rows available. Increase rows selector to view more rows.	

Figure 64: Show Specific Data from Table "employees"

```
SELECT LAST_NAME, SALARY
FROM employees
WHERE SALARY IN (10000,20000,30000);
```

Results	Explain	Describe	Saved SQL	History
LAST_NAME		SALARY		
Tucker		10000		
King		10000		
Bloom		10000		
Baer		10000		

4 rows returned in 0.00 seconds [CSV Export](#)

Figure 65: Show Specific Data from Table "employees"

```

SELECT FIRST_NAME
FROM employees
WHERE FIRST_NAME LIKE 'S%';

```

Results	Explain	Describe	Saved SQL	History
FIRST_NAME				
Sundar				
Shelli				
Sarah				
Shelley				
Steven				
Sundita				
Steven				
Susan				
Samuel				
Sarath				
More than 10 rows available. Increase rows selector to view more rows.				
10 rows returned in 0.00 seconds CSV Export				

Figure 66: Values Where FIRST_NAME Starts with S

```

SELECT FIRST_NAME
FROM employees
WHERE FIRST_NAME LIKE '_o%';

```

Results	Explain	Describe	Saved SQL	History
FIRST_NAME				
Mozhe				
John				
Louise				
Douglas				
Donald				
Joshua				
John				
John				
Jonathon				
Jose Manuel				

Figure 67: Show Specific Data from Table "employees"

```
SELECT LAST_NAME, manager_id
FROM employees
WHERE manager_ID IS NULL;
```

Results

Explain

Describe

Saved SQL

History

LAST_NAME

MANAGER_ID

King

-

1 rows returned in 0.02 seconds

[CSV Export](#)

Figure 68: Show Specific Data from Table "employees"

```
SELECT LAST_NAME, manager_id
FROM employees
WHERE COMMISSION_PCT IS NULL;
```

Results

Explain

Describe

Saved SQL

History

LAST_NAME	MANAGER_ID
King	-
Kochhar	100
De Haan	100
Hunold	102
Ernst	103
Austin	103
Pataballa	103
Lorentz	103
Greenberg	101
Faviet	108
More than 10 rows available. Increase rows selector to view more rows.	

10 rows returned in 0.00 seconds

[CSV Export](#)

Figure 69: Show Specific Data from Table "employees"

```

SELECT MANAGER_ID
FROM employees
WHERE HIRE_DATE BETWEEN TO_DATE('21-SEP-89', 'DD-MON-YY')
                        AND TO_DATE('16-AUG-94', 'DD-MON-YY');

```

Results Explain Describe Saved SQL History

no data found

Figure 70: Result of Selected Data

```

SELECT FIRST_NAME
FROM employees
WHERE LAST_NAME IN ('Jake', 'Lee', 'Kumar');

```

Results Explain Describe Saved SQL History

FIRST_NAME
Sundita
David

2 rows returned in 0.00 seconds

[CSV Export](#)

Figure 71: Show Specific Data from Table "employees"

```

SELECT employee_id, last_name, JOB_ID, salary
FROM employees
WHERE salary >=10000
AND JOB_ID Like '%MAN%';

```

Results Explain Describe Saved SQL History

EMPLOYEE_ID	LAST_NAME	JOB_ID	SALARY
114	Raphaely	PU_MAN	11000
145	Russell	SA_MAN	14000
146	Partners	SA_MAN	13500
147	Errazuriz	SA_MAN	12000
148	Cambrault	SA_MAN	11000
149	Zlotkey	SA_MAN	10500
201	Hartstein	MK_MAN	13000

7 rows returned in 0.01 seconds

[CSV Export](#)

Figure 72: Show Specific Data from Table "employees"

```

SELECT employee_id
FROM employees
WHERE SALARY > 200
AND JOB_ID LIKE '%IT_PROG%';

```

Results Explain Describe Saved SQL History

EMPLOYEE_ID
103
104
105
106
107

5 rows returned in 0.00 seconds

[CSV Export](#)

Figure 73: Show Specific Data from Table "employees"

```

SELECT employee_id
FROM employees
WHERE SALARY > 200
OR JOB_ID LIKE '%IT_PROG%';

```

Results

Explain

Describe

Saved SQL

History

EMPLOYEE_ID

100

101

102

103

104

105

106

107

108

109

More than 10 rows available. Increase rows selector to view more rows.

Figure 74: Show Specific Data from Table "employees"

```

SELECT LAST_NAME, JOB_ID
FROM employees
WHERE JOB_ID
NOT IN ('IT_PROG', 'ST_CLEAR', 'SA_REP');

```

Results

Explain

Describe

Saved SQL

History

LAST_NAME

JOB_ID

King

AD_PRES

Kochhar

AD_VP

De Haan

AD_VP

Greenberg

FI_MGR

Faviet

FI_ACCOUNT

Chen

FI_ACCOUNT

Sciarra

FI_ACCOUNT

Urman

FI_ACCOUNT

Popp

FI_ACCOUNT

Raphaely

PU_MAN

More than 10 rows available. Increase rows selector to view more rows.

Figure 75: Show Specific Data from Table "employees"

```

SELECT employee_id, last_name, JOB_ID, salary
FROM employees
WHERE JOB_ID Like '%VP%';

```

Results	Explain	Describe	Saved SQL	History
EMPLOYEE_ID	LAST_NAME	JOB_ID	SALARY	
101	Kochhar	AD_VP	17000	
102	De Haan	AD_VP	17000	
2 rows returned in 0.00 seconds				CSV Export

Figure 76: Show Specific Data from Table "employees"

```

SELECT LAST_NAME, JOB_ID, SALARY
FROM employees
WHERE JOB_ID Like 'SA_REP'
OR JOB_ID = 'AD_PRES'
AND SALARY > 15000;

```

Results	Explain	Describe	Saved SQL	History
LAST_NAME	JOB_ID	SALARY		
King	AD_PRES	24000		
Tucker	SA_REP	10000		
Bernstein	SA_REP	9500		
Hall	SA_REP	9000		
Olsen	SA_REP	8000		
Cambrault	SA_REP	7500		
Tuvault	SA_REP	7000		
King	SA_REP	10000		
Sully	SA_REP	9500		
McEwen	SA_REP	9000		
More than 10 rows available. Increase rows selector to view more rows.				
10 rows returned in 0.00 seconds				CSV Export

Figure 77: Show Specific Data from Table "employees"

```

SELECT LAST_NAME, JOB_ID, SALARY
FROM employees
WHERE (JOB_ID Like 'SA_REP'
OR JOB_ID = 'AD_PRES')
AND SALARY > 15000;

```

Results

Explain

Describe

Saved SQL

History

LAST_NAME

JOB_ID

SALARY

King

AD_PRES

24000

1 rows returned in 0.00 seconds

[CSV Export](#)

Figure 78: Show Specific Data from Table "employees"

```

SELECT LAST_NAME, JOB_ID, DEPARTMENT_ID, HIRE_DATE
FROM employees
ORDER BY SALARY DESC

```

Results

Explain

Describe

Saved SQL

History

LAST_NAME	JOB_ID	DEPARTMENT_ID	HIRE_DATE
King	AD_PRES	90	17-JUN-87
Kochhar	AD_VP	90	21-SEP-89
De Haan	AD_VP	90	13-JAN-93
Russell	SA_MAN	80	01-OCT-96
Partners	SA_MAN	80	05-JAN-97
Hartstein	MK_MAN	20	17-FEB-96
Greenberg	FI_MGR	100	17-AUG-94
Errazuriz	SA_MAN	80	10-MAR-97
Higgins	AC_MGR	110	07-JUN-94
Ozer	SA_REP	80	11-MAR-97
More than 10 rows available. Increase rows selector to view more rows.			

10 rows returned in 0.00 seconds

CSV Export

Figure 79: Show Specific Data from Table "employees"

LAB 6: OTHER RESTRICTING DATA CONSTRAINTS

Use University Database

USE university;

1. Filtering Data Using Multiple Conditions

1.1 Create Table Students

```
CREATE TABLE students (
    id INT PRIMARY KEY,
    name VARCHAR(50),
    age INT,
    city VARCHAR(50),
    grade CHAR(1)
);
```

[Results](#)
[Explain](#)
[Describe](#)
[Saved SQL](#)
[History](#)

Object Type

TABLE

Object

STUDENTS

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
STUDENTS	ID	Number	-	-	0	1	-	-	-
	NAME	Varchar2	50	-	-	-	✓	-	-
	AGE	Number	-	-	0	-	✓	-	-
	CITY	Varchar2	50	-	-	-	✓	-	-
	GRADE	Char	1	-	-	-	✓	-	-
1 - 5									

Figure 80: Table Created

Figure 80: Table Created

1.2 Insert Data

INSERT ALL

```
    INTO students (id, name, age, city, grade) VALUES (1, 'Aisha Khan',
20, 'New York', 'B')
```

```
    INTO students (id, name, age, city, grade) VALUES (2, 'Fatima Ali',
22, 'Los Angeles', 'A')
```

```
    INTO students (id, name, age, city, grade) VALUES (3, 'Zainab
Malik', 19, 'Chicago', 'B')
```

```

    INTO students (id, name, age, city, grade) VALUES (4, 'Maryam
Siddiqui', 21, 'New York', 'A')

    INTO students (id, name, age, city, grade) VALUES (5, 'Noor Ahmed',
23, 'Chicago', 'C')

    INTO students (id, name, age, city, grade) VALUES (6, 'Huda Rahman',
20, 'Los Angeles', 'B')

    INTO students (id, name, age, city, grade) VALUES (7, 'Layla
Hussain', 22, 'Houston', 'A')

    INTO students (id, name, age, city, grade) VALUES (8, 'Iman Yusuf',
21, 'New York', 'A')

    INTO students (id, name, age, city, grade) VALUES (9, 'Nadia Karim',
20, 'Houston', 'B')

    INTO students (id, name, age, city, grade) VALUES (10, 'Rania Omar',
24, 'Miami', 'C')

SELECT * FROM dual;

```

Results	Explain	Describe	Saved SQL	History
ID	NAME	AGE	CITY	GRADE
1	Aisha Khan	20	New York	B
2	Fatima Ali	22	Los Angeles	A
3	Zainab Malik	19	Chicago	B
4	Maryam Siddiqui	21	New York	A
5	Noor Ahmed	23	Chicago	C
6	Huda Rahman	20	Los Angeles	B
7	Layla Hussain	22	Houston	A
8	Iman Yusuf	21	New York	A
9	Nadia Karim	20	Houston	B
10	Rania Omar	24	Miami	C

10 rows returned in 0.00 seconds [CSV Export](#)

Figure 81: Data Inserted

1.2 Students from New York with grade 'A'

```
SELECT * FROM students
WHERE city = 'New York' AND grade = 'A';
```

Results	Explain	Describe	Saved SQL	History
ID	NAME	AGE	CITY	GRADE
4	Maryam Siddiqui	21	New York	A
8	Iman Yusuf	21	New York	A

2 rows returned in 0.00 seconds [CSV Export](#)

Figure 82: Data Displayed

1.3 Students from Los Angeles or with grade 'B'

```
SELECT * FROM students
WHERE city = 'Los Angeles' OR grade = 'B';
```

Results	Explain	Describe	Saved SQL	History
ID	NAME	AGE	CITY	GRADE
1	Aisha Khan	20	New York	B
2	Fatima Ali	22	Los Angeles	A
3	Zainab Malik	19	Chicago	B
6	Huda Rahman	20	Los Angeles	B
9	Nadia Karim	20	Houston	B

5 rows returned in 0.02 seconds [CSV Export](#)

Figure 83: Table Displayed

1.4 Students not from Chicago

```
SELECT * FROM students
WHERE NOT city = 'Chicago';
```

Results	Explain	Describe	Saved SQL	History
ID	NAME	AGE	CITY	GRADE
1	Aisha Khan	20	New York	B
2	Fatima Ali	22	Los Angeles	A
4	Maryam Siddiqui	21	New York	A
6	Huda Rahman	20	Los Angeles	B
7	Layla Hussain	22	Houston	A
8	Iman Yusuf	21	New York	A
9	Nadia Karim	20	Houston	B
10	Rania Omar	24	Miami	C

8 rows returned in 0.00 seconds [CSV Export](#)

Figure 84: Table Displayed

2. Using Aggregate Functions

2.1 Total number of students

```
SELECT COUNT(*) AS total_students
FROM students;
```

Results	Explain	Describe	Saved SQL	History
TOTAL_STUDENTS				
10				

1 rows returned in 0.00 seconds [CSV Export](#)

Figure 85: Data Displayed

2.2 Average age of students

```
SELECT AVG(age) AS average_age
FROM students;
```

Results	Explain	Describe	Saved SQL	History
AVERAGE_AGE				
21.2				
1 rows returned in 0.00 seconds				
CSV Export				

Figure 86: Data Displayed

2.3 Youngest and oldest students

```
SELECT MIN(age) AS youngest, MAX(age) AS oldest
FROM students;
```

Results	Explain	Describe	Saved SQL	History
YOUNGEST OLDEST				
19 24				
1 rows returned in 0.01 seconds				
CSV Export				

Figure 87: Age Displayed

2.4 Number of students in each city

```
SELECT city, COUNT(*) AS student_count
FROM students
GROUP BY city;
```

Results	Explain	Describe	Saved SQL	History
CITY STUDENT_COUNT				
New York 3				
Los Angeles 2				
Chicago 2				
Miami 1				
Houston 2				
5 rows returned in 0.01 seconds				
CSV Export				

Figure 88: Table Displayed

3. Grouping and Filtering Aggregated Data

3.1 Cities with more than one student

```
SELECT city, COUNT(*) AS student_count
FROM students
GROUP BY city
HAVING COUNT(*) > 1;
```

Results

Explain

Describe

Saved SQL

History

CITY	STUDENT_COUNT
New York	3
Los Angeles	2
Chicago	2
Houston	2

4 rows returned in 0.00 seconds

[CSV Export](#)

Figure 89: Table Displayed

Figure 89: Table Displayed

3.2 Average age for each grade

```
SELECT grade, AVG(age) AS average_age
FROM students
GROUP BY grade;
```

Results

Explain

Describe

Saved SQL

History

GRADE	AVERAGE_AGE
B	19.75
A	21.5
C	23.5

3 rows returned in 0.01 seconds

[CSV Export](#)

Figure 90: Table Displayed

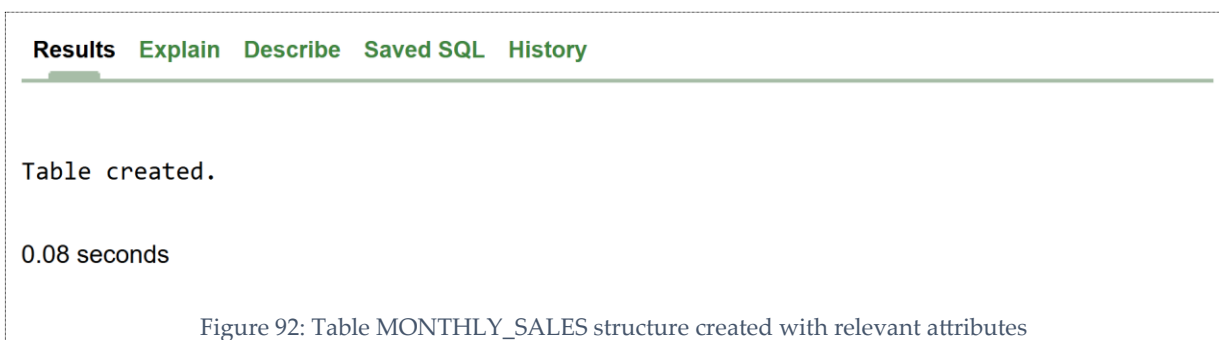
Figure 90: Table Displayed

LAB 7: PRACTICAL TEST (CREATE, ALTER, DROP, INSERT, UPDATE, DELETE)

Task 1

Monthly Sales Table

```
CREATE TABLE MONTHLY_SALES (
    CUST_ID NUMBER,
    PROD_ID NUMBER,
    PROD_NAME VARCHAR2(50),
    PRICE NUMBER(10,2),
    BILL_TOTAL DECIMAL(10,2),
    SALE_DATE DATE
);
```



1.1 Inserting Values into the table

INSERT ALL

```
    INTO MONTHLY_SALES (CUST_ID, PROD_ID, PROD_NAME, PRICE,
BILL_TOTAL, SALE_DATE)
    VALUES (1, 1, 'Shampoo', 165, 2, TO_DATE('26-Mar-23','DD-Mon-YY'))
    INTO MONTHLY_SALES (CUST_ID, PROD_ID, PROD_NAME, PRICE,
BILL_TOTAL, SALE_DATE)
    VALUES (2, 2, 'Soap', 32, 10, TO_DATE('26-Mar-23','DD-Mon-YY'))
    INTO MONTHLY_SALES (CUST_ID, PROD_ID, PROD_NAME, PRICE,
BILL_TOTAL, SALE_DATE)
    VALUES (3, 3, 'Cream', 85, 1, TO_DATE('26-Mar-23','DD-Mon-YY'))
```

```

    INTO MONTHLY_SALES (CUST_ID, PROD_ID, PROD_NAME, PRICE,
BILL_TOTAL, SALE_DATE)

    VALUES (4, 1, 'Shampoo', 165, 1, TO_DATE('27-Mar-23','DD-Mon-YY'))

    INTO MONTHLY_SALES (CUST_ID, PROD_ID, PROD_NAME, PRICE,
BILL_TOTAL, SALE_DATE)

    VALUES (5, 3, 'Cream', 85, 2, TO_DATE('28-Mar-23','DD-Mon-YY'))

SELECT * FROM dual;

```

Results

Explain

Describe

Saved SQL

History

CUST_ID	PROD_ID	PROD_NAME	PRICE	BILL_TOTAL	SALE_DATE
1	1	Shampoo	165	2	26-MAR-23
2	2	Soap	32	10	26-MAR-23
3	3	Cream	85	1	26-MAR-23
4	1	Shampoo	165	1	27-MAR-23
5	3	Cream	85	2	28-MAR-23

5 rows returned in 0.00 seconds

CSV Export

Figure 93: Records inserted into MONTHLY_SALES using INSERT ALL statement

1.2 Adding new Coloumn

```
ALTER TABLE MONTHLY_SALES
```

```
ADD Return_Date DATE;
```

Results

Explain

Describe

Saved SQL

History

CUST_ID	PROD_ID	PROD_NAME	PRICE	BILL_TOTAL	SALE_DATE	RETURN_DATE
1	1	Shampoo	165	2	26-MAR-23	-
2	2	Soap	32	10	26-MAR-23	-
3	3	Cream	85	1	26-MAR-23	-
4	1	Shampoo	165	1	27-MAR-23	-
5	3	Cream	85	2	28-MAR-23	-

5 rows returned in 0.00 seconds

CSV Export

Figure 94: "Return_Date" Added

1.3 Drop Column using ALTER Command

```
ALTER TABLE MONTHLY_SALES
```

```
DROP COLUMN RETURN_DATE
```

Results

Explain

Describe

Saved SQL

History

CUST_ID	PROD_ID	PROD_NAME	PRICE	BILL_TOTAL	SALE_DATE
1	1	Shampoo	165	2	26-MAR-23
2	2	Soap	32	10	26-MAR-23
3	3	Cream	85	1	26-MAR-23
4	1	Shampoo	165	1	27-MAR-23
5	3	Cream	85	2	28-MAR-23

5 rows returned in 0.00 seconds

[CSV Export](#)

Figure 95: Column Dropped

1.4 Update Column using UPDATE Command

```
UPDATE MONTHLY_SALES
```

```
SET Price = 100
```

```
WHERE PROD_ID = 3;
```

Results

Explain

Describe

Saved SQL

History

CUST_ID	PROD_ID	PROD_NAME	PRICE	BILL_TOTAL	SALE_DATE
1	1	Shampoo	165	2	26-MAR-23
2	2	Soap	32	10	26-MAR-23
3	3	Cream	100	1	26-MAR-23
4	1	Shampoo	165	1	27-MAR-23
5	3	Cream	100	2	28-MAR-23

5 rows returned in 0.00 seconds

[CSV Export](#)

Figure 96: Data Updated

Task 2:

Student Table

```
CREATE TABLE STUDENT (
    Name VARCHAR2(50),
    Address VARCHAR2(100),
    Class VARCHAR2(20),
    CGPA NUMBER(3,2)
);
```

Results

Explain

Describe

Saved SQL

History

Object Type

TABLE

Object

STUDENT

Table	Column	Data Type	Length	Precision	Scale	Primary Key	Nullable	Default	Comment
STUDENT	NAME	Varchar2	50	-	-	-	✓	-	-
	ADDRESS	Varchar2	100	-	-	-	✓	-	-
	CLASS	Varchar2	20	-	-	-	✓	-	-
	CGPA	Number	-	3	2	-	✓	-	-
									1 - 4

Figure 97: "Student" Table Structure

1.1 Insert Data

```
INSERT ALL
INTO STUDENT VALUES ('Humna', 'Karachi', 'BSAI', 3.85)
INTO STUDENT VALUES ('Zainab', 'Lahore', 'BSCS', 3.72)
INTO STUDENT VALUES ('Fatima', 'Islamabad', 'BSIT', 3.68)
SELECT * FROM DUAL;
```

Results

Explain

Describe

Saved SQL

History

NAME	ADDRESS	CLASS	CGPA
Humna	Karachi	BSAI	3.85
Zainab	Lahore	BSCS	3.72
Fatima	Islamabad	BSIT	3.68

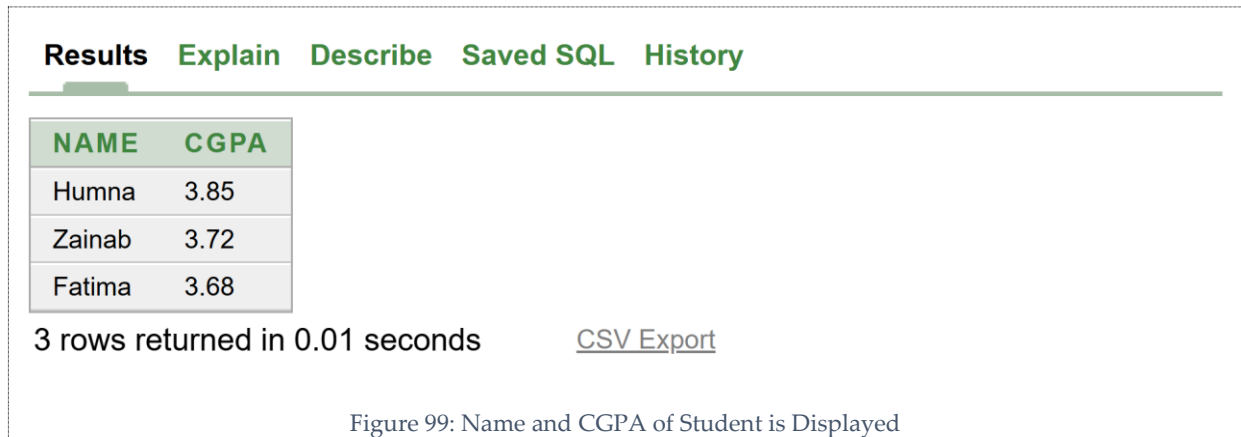
3 rows returned in 0.00 seconds

[CSV Export](#)

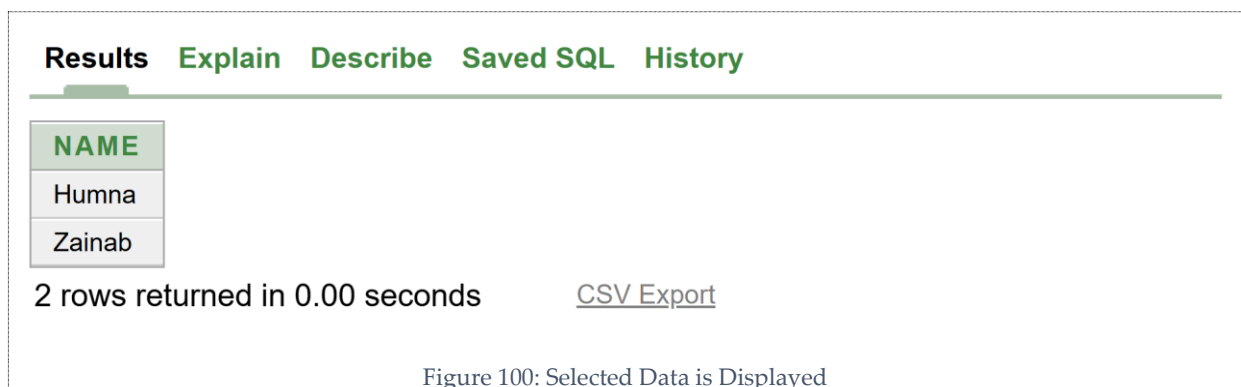
Figure 98: Data Entered

1.2 Display Selected Data Only

```
SELECT Name, CGPA
FROM STUDENT;
```



```
SELECT Name
FROM STUDENT
WHERE CGPA > 3.7;
```



LAB 8: DATABASE SETUP USING XAMPP

INTRODUCTION TO MySQL(XAMPP)

Databases

Create database ⓘ

Figure 101: Launching MySQL module in XAMPP Control Panel

Built-in:

☐ Check all

Database ▲	Collation	Action
☐ information_schema	utf8_general_ci	Check privileges
☐ mysql	utf8mb4_general_ci	Check privileges
☐ performance_schema	utf8_general_ci	Check privileges
☐ phpmyadmin	utf8_bin	Check privileges
☐ test	latin1_swedish_ci	Check privileges
Total: 5		

Figure 102: Accessing phpMyAdmin for executing MySQL queries

Creating new table.

Create new table

Table name

Number of columns

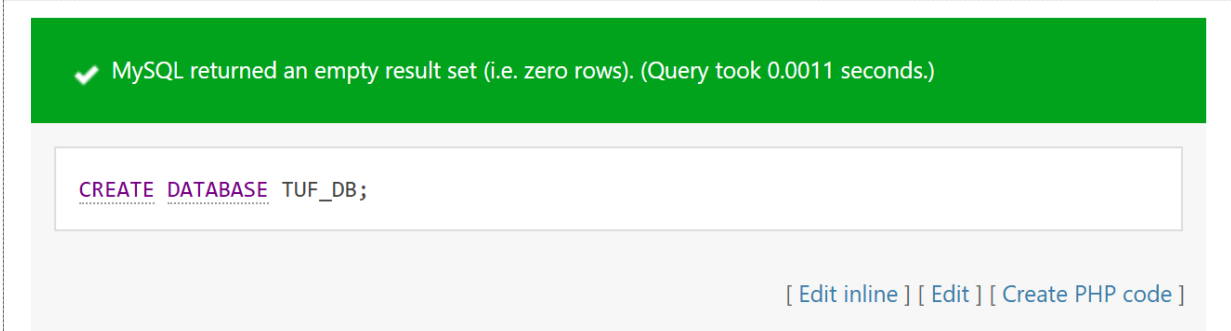
Figure 103: Creating a new table using phpMyAdmin interface

CREATE DATABASE AMAZONE

SHOW DATABASES

DROP DATABASE AMAZONE

CREATE DATABASE TUF_DB



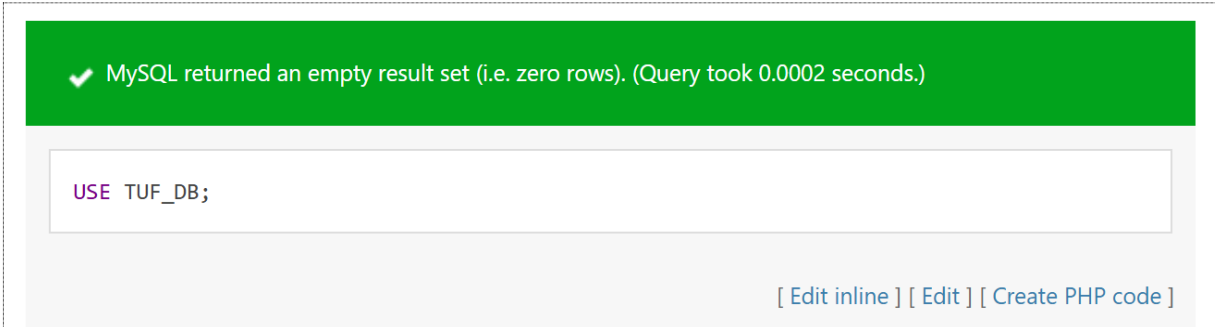
✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0011 seconds.)

```
CREATE DATABASE TUF_DB;
```

[\[Edit inline \]](#) [\[Edit \]](#) [\[Create PHP code \]](#)

Figure 104: Creating AMAZONE database using CREATE DATABASE command

USE TUF_DB



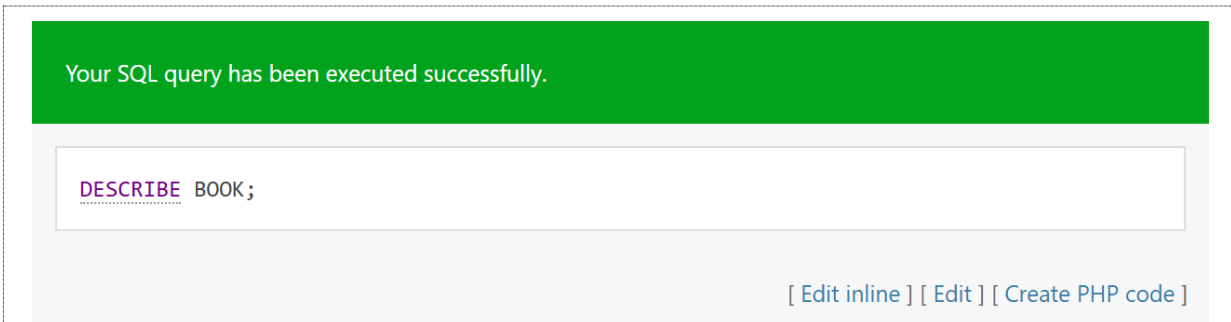
✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0002 seconds.)

```
USE TUF_DB;
```

[\[Edit inline \]](#) [\[Edit \]](#) [\[Create PHP code \]](#)

Figure 105: Using TUF_DB database for further operations

DESCRIBE BOOK



Your SQL query has been executed successfully.

```
DESCRIBE BOOK;
```

[\[Edit inline \]](#) [\[Edit \]](#) [\[Create PHP code \]](#)

Figure 106: Table BOOK described using DESCRIBE command

ALTER TABLE BOOK;

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0005 seconds.)

```
ALTER TABLE BOOK;
```

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

Figure 107: Altering BOOK table to add a new column PRICE

ALTER TABLE BOOK

ADD PRICE INT;

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0006 seconds.)

```
ALTER TABLE BOOK ADD PRICE INT;
```

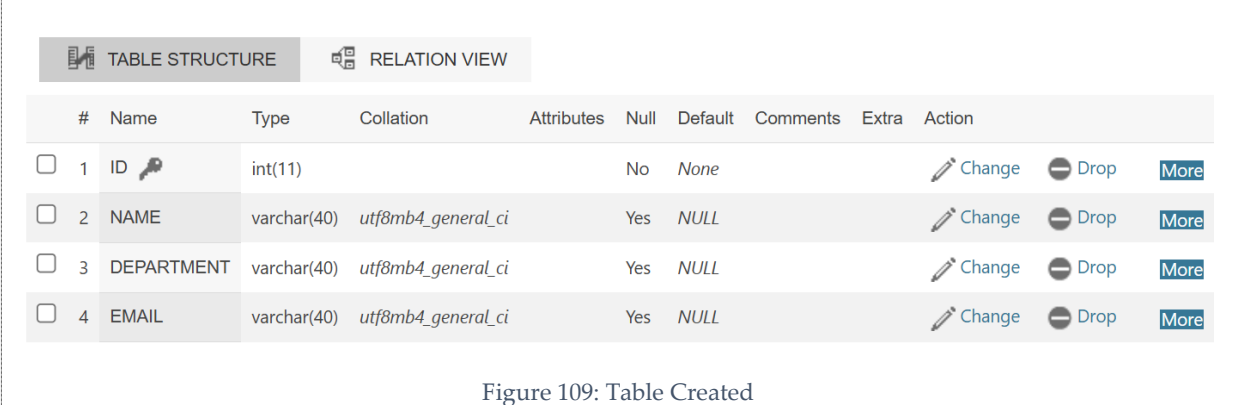
[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

Figure 108: Altering BOOK table to add a new column PRICE

LAB 9: MYSQL DDL QUERIES USING DIFFERENT SCENARIOS

1. Create “ITFACULTY”

```
CREATE TABLE ITFACULTY(
    ID VARCHAR(100) PRIMARY KEY,
    NAME VARCHAR(100),
    DEPARTMENT VARCHAR(100),
    EMAIL VARCHAR(100));
```



The screenshot shows the 'TABLE STRUCTURE' tab for the 'ITFACULTY' table. It displays four columns: ID, NAME, DEPARTMENT, and EMAIL. The 'ID' column is the primary key. The 'NAME', 'DEPARTMENT', and 'EMAIL' columns are of type VARCHAR(40) with a collation of 'utf8mb4_general_ci'. The 'ID' column is of type int(11). The 'Null' column for 'ID' is 'No', while for the others it is 'Yes'. The 'Default' column for 'ID' is 'None', while for the others it is 'NULL'. The 'Action' column for each row contains 'Change', 'Drop', and 'More' links.

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	ID	int(11)			No	None			Change Drop More
2	NAME	varchar(40)	utf8mb4_general_ci		Yes	NULL			Change Drop More
3	DEPARTMENT	varchar(40)	utf8mb4_general_ci		Yes	NULL			Change Drop More
4	EMAIL	varchar(40)	utf8mb4_general_ci		Yes	NULL			Change Drop More

Figure 109: Table Created

2. Create “STD_BSAI”

```
CREATE TABLE STD_BSAI(
    REG_NO VARCHAR(50),
    STD_NAME VARCHAR(40),
    DOB DATE,
    STD_PHONE VARCHAR(10),
    STD_ADDRESS VARCHAR(70),
    FACULTY_ID INT,
    FOREIGN KEY (FACULTY_ID)
        REFERENCES IT_FACULTY(ID)
);
```

TABLE STRUCTURE									
#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1	REG_NO	varchar(50) utf8mb4_general_ci		Yes	NULL			Change Drop More
☐	2	STD_NAME	varchar(40) utf8mb4_general_ci		Yes	NULL			Change Drop More
☐	3	DOB	date		Yes	NULL			Change Drop More
☐	4	STD_PHONE	varchar(10) utf8mb4_general_ci		Yes	NULL			Change Drop More
☐	5	STD_ADDRESS	varchar(70) utf8mb4_general_ci		Yes	NULL			Change Drop More
☐	6	FACULTY_ID	int(11)		Yes	NULL			Change Drop More

Figure 110: Table Created

2.1 Add New Column “EMAIL”

ALTER TABLE STD_BSAI

ADD EMAIL VARCHAR(40)

TABLE STRUCTURE									
#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1	REG_NO	varchar(50) utf8mb4_general_ci		Yes	NULL			Change Drop More
☐	2	STD_NAME	varchar(40) utf8mb4_general_ci		Yes	NULL			Change Drop More
☐	3	DOB	date		Yes	NULL			Change Drop More
☐	4	STD_PHONE	varchar(10) utf8mb4_general_ci		Yes	NULL			Change Drop More
☐	5	STD_ADDRESS	varchar(70) utf8mb4_general_ci		Yes	NULL			Change Drop More
☐	6	FACULTY_ID	int(11)		Yes	NULL			Change Drop More
☐	7	EMAIL	varchar(40) utf8mb4_general_ci		Yes	NULL			Change Drop More

Figure 111: New Column Added

LAB 10: MYSQL DML QUERIES USING DIFFERENT SCENARIOS

1. Monthly Sales Table

1.1 Create Table

```
CREATE TABLE MONTHLY_SALES (
  PROD_ID INT PRIMARY KEY,
  PROD_NAME VARCHAR(50) NOT NULL,
  PRICE INT UNIQUE,
  QUANTITY INT,
  BILL_TOTAL INT,
  DOB DATE
);
```

TABLE STRUCTURE									
#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐ 1	PROD_ID	int(11)			No	None			Change Drop More
☐ 2	PROD_NAME	varchar(50)	latin1_swedish_ci		No	None			Change Drop More
☐ 3	PRICE	int(11)			Yes	NULL			Change Drop More
☐ 4	QUANTITY	int(11)			Yes	NULL			Change Drop More
☐ 5	BILL_TOTAL	int(11)			Yes	NULL			Change Drop More
☐ 6	DOB	date			Yes	NULL			Change Drop More

Figure 112: Table Created

1.2 Insert Values into Table

```
INSERT INTO monthly_sales (PROD_ID, PROD_NAME, PRICE, QUANTITY,
BILL_TOTAL, DOB) VALUES
(1, 'Apple iPhone', 999, 2, 1998, '2024-12-01'),
(2, 'Samsung Galaxy', 850, 1, 850, '2024-11-15'),
```

```
(3, 'Google Pixel', 799, 3, 2397, '2024-10-20'),
(4, 'OnePlus Nord', NULL, NULL, NULL, NULL),
(5, 'Sony Xperia', 650, 2, 1300, '2024-08-05'),
(6, 'Nokia Lumia', 300, 5, 1500, '2024-07-01'),
(7, 'Motorola Edge', 550, 1, 550, '2024-06-12'),
(8, 'Xiaomi Redmi', 400, 4, 1600, '2024-05-25'),
(9, 'Realme Narzo', 350, 2, 700, '2024-04-14'),
(10, 'Asus Zenfone', 600, 3, 1800, '2024-03-30');
```

←T→	PROD_ID	PROD_NAME	PRICE	QUANTITY	BILL_TOTAL	DOB
☐  Edit  Copy  Delete	1	Apple iPhone	999	2	1998	2024-12-01
☐  Edit  Copy  Delete	2	Samsung Galaxy	850	1	850	2024-11-15
☐  Edit  Copy  Delete	3	Google Pixel	799	3	2397	2024-10-20
☐  Edit  Copy  Delete	4	OnePlus Nord	NULL	NULL	NULL	NULL
☐  Edit  Copy  Delete	5	Sony Xperia	650	2	1300	2024-08-05
☐  Edit  Copy  Delete	6	Nokia Lumia	300	5	1500	2024-07-01
☐  Edit  Copy  Delete	7	Motorola Edge	550	1	550	2024-06-12
☐  Edit  Copy  Delete	8	Xiaomi Redmi	400	4	1600	2024-05-25
☐  Edit  Copy  Delete	9	Realme Narzo	350	2	700	2024-04-14
☐  Edit  Copy  Delete	10	Asus Zenfone	600	3	1800	2024-03-30

Figure 113: Rows Inserted

1.3 Adding new Column

```
ALTER TABLE MONTHLY_SALES
ADD COLUMN Return_Date DATE;
```

←T→				PROD_ID	PROD_NAME	PRICE	QUANTITY	BILL_TOTAL	DOB	Return_Date
☐	 Edit	 Copy	 Delete	1	Apple iPhone	999	2	1998	2024-12-01	NULL
☐	 Edit	 Copy	 Delete	2	Samsung Galaxy	850	1	850	2024-11-15	NULL
☐	 Edit	 Copy	 Delete	3	Google Pixel	799	3	2397	2024-10-20	NULL
☐	 Edit	 Copy	 Delete	4	OnePlus Nord	NULL	NULL	NULL	NULL	NULL
☐	 Edit	 Copy	 Delete	5	Sony Xperia	650	2	1300	2024-08-05	NULL
☐	 Edit	 Copy	 Delete	6	Nokia Lumia	300	5	1500	2024-07-01	NULL
☐	 Edit	 Copy	 Delete	7	Motorola Edge	550	1	550	2024-06-12	NULL
☐	 Edit	 Copy	 Delete	8	Xiaomi Redmi	400	4	1600	2024-05-25	NULL
☐	 Edit	 Copy	 Delete	9	Realme Narzo	350	2	700	2024-04-14	NULL
☐	 Edit	 Copy	 Delete	10	Asus Zenfone	600	3	1800	2024-03-30	NULL

Figure 114: New Column Added

1.4 Rename Using ALTER Query

ALTER TABLE MONTHLY_SALES

CHANGE COLUMN DOB Sales_Date DATE;

←T→				PROD_ID	PROD_NAME	PRICE	QUANTITY	BILL_TOTAL	Sales_Date	Return_Date
☐	 Edit	 Copy	 Delete	1	Apple iPhone	999	2	1998	2024-12-01	NULL
☐	 Edit	 Copy	 Delete	2	Samsung Galaxy	850	1	850	2024-11-15	NULL
☐	 Edit	 Copy	 Delete	3	Google Pixel	799	3	2397	2024-10-20	NULL
☐	 Edit	 Copy	 Delete	4	OnePlus Nord	NULL	NULL	NULL	NULL	NULL
☐	 Edit	 Copy	 Delete	5	Sony Xperia	650	2	1300	2024-08-05	NULL
☐	 Edit	 Copy	 Delete	6	Nokia Lumia	300	5	1500	2024-07-01	NULL
☐	 Edit	 Copy	 Delete	7	Motorola Edge	550	1	550	2024-06-12	NULL
☐	 Edit	 Copy	 Delete	8	Xiaomi Redmi	400	4	1600	2024-05-25	NULL
☐	 Edit	 Copy	 Delete	9	Realme Narzo	350	2	700	2024-04-14	NULL
☐	 Edit	 Copy	 Delete	10	Asus Zenfone	600	3	1800	2024-03-30	NULL

Figure 115: Column Renamed

1.5 Drop Column Using DROP Query

```
ALTER TABLE MONTHLY_SALES
```

```
DROP COLUMN Return_Date;
```

←T→		PROD_ID	PROD_NAME	PRICE	QUANTITY	BILL_TOTAL	Sales_Date
☐	 Edit  Copy  Delete	1	Apple iPhone	999	2	1998	2024-12-01
☐	 Edit  Copy  Delete	2	Samsung Galaxy	850	1	850	2024-11-15
☐	 Edit  Copy  Delete	3	Google Pixel	799	3	2397	2024-10-20
☐	 Edit  Copy  Delete	4	OnePlus Nord	NULL	NULL	NULL	NULL
☐	 Edit  Copy  Delete	5	Sony Xperia	650	2	1300	2024-08-05
☐	 Edit  Copy  Delete	6	Nokia Lumia	300	5	1500	2024-07-01
☐	 Edit  Copy  Delete	7	Motorola Edge	550	1	550	2024-06-12
☐	 Edit  Copy  Delete	8	Xiaomi Redmi	400	4	1600	2024-05-25
☐	 Edit  Copy  Delete	9	Realme Narzo	350	2	700	2024-04-14
☐	 Edit  Copy  Delete	10	Asus Zenfone	600	3	1800	2024-03-30

Figure 116: Column Dropped

1.6 Update Column using UPDATE Command

```
UPDATE MONTHLY_SALES
```

```
SET Price = 100
```

```
WHERE PROD_ID = 3;
```

← T →				PROD_ID	PROD_NAME	PRICE	QUANTITY	BILL_TOTAL	Sales_Date
☐	Edit	Copy	Delete	1	Apple iPhone	999	2	1998	2024-12-01
☐	Edit	Copy	Delete	2	Samsung Galaxy	850	1	850	2024-11-15
☐	Edit	Copy	Delete	3	Google Pixel	100	3	2397	2024-10-20
☐	Edit	Copy	Delete	4	OnePlus Nord	NULL	NULL	NULL	NULL
☐	Edit	Copy	Delete	5	Sony Xperia	650	2	1300	2024-08-05
☐	Edit	Copy	Delete	6	Nokia Lumia	300	5	1500	2024-07-01
☐	Edit	Copy	Delete	7	Motorola Edge	550	1	550	2024-06-12
☐	Edit	Copy	Delete	8	Xiaomi Redmi	400	4	1600	2024-05-25
☐	Edit	Copy	Delete	9	Realme Narzo	350	2	700	2024-04-14
☐	Edit	Copy	Delete	10	Asus Zenfone	600	3	1800	2024-03-30

Figure 117: Column Updated

2. Student Table

2.1 “CREATE” Command

```
CREATE TABLE STUDENT (
  NAME VARCHAR(50),
  ADDRESS VARCHAR(100),
  CLASS VARCHAR(10),
  CGPA DECIMAL(3,2)
);
```

TABLE STRUCTURE

RELATION VIEW

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐	1	Name	varchar(50)	latin1_swedish_ci		Yes	NULL		Change Drop More
☐	2	Address	varchar(100)	latin1_swedish_ci		Yes	NULL		Change Drop More
☐	3	Class	varchar(10)	latin1_swedish_ci		Yes	NULL		Change Drop More
☐	4	CGPA	decimal(3,2)			Yes	NULL		Change Drop More

Figure 118: Table Created

2.2 Insert Values in Table

```
INSERT INTO student (Name, Address, Class, CGPA) VALUES
('Aisha Khan', 'Makkah', '10A', 3.85),
('Fatima Noor', 'Madinah', '9B', 3.70),
('Zainab Ahmed', 'Jerusalem', '11A', 3.95),
('Maryam Siddiqui', 'Karbala', '12B', 3.60),
('Hafsa Rahman', 'Istanbul', '10C', 3.45),
('Sumayya Ali', 'Cairo', '11B', 3.55),
('Asma Yusuf', 'Damascus', '9A', 3.30),
('Ruqayyah Hasan', 'Baghdad', '12A', 3.90),
('Aaliyah Karim', 'Najaf', '10B', 3.40),
('Nusaybah Omar', 'Al-Quds', '11C', 3.75);
```

Name	Address	Class	CGPA
Aisha Khan	Makkah	10A	3.85
Fatima Noor	Madinah	9B	3.70
Zainab Ahmed	Jerusalem	11A	3.95
Maryam Siddiqui	Karbala	12B	3.60
Hafsa Rahman	Istanbul	10C	3.45
Sumayya Ali	Cairo	11B	3.55
Asma Yusuf	Damascus	9A	3.30
Ruqayyah Hasan	Baghdad	12A	3.90
Aaliyah Karim	Najaf	10B	3.40
Nusaybah Omar	Al-Quds	11C	3.75

Figure 119: Values Inserted

2.3 Display Student Names and CGPA Only

```
SELECT Name, CGPA FROM STUDENT;
```

Name	CGPA
Aisha Khan	3.85
Fatima Noor	3.70
Zainab Ahmed	3.95
Maryam Siddiqui	3.60
Hafsa Rahman	3.45
Sumayya Ali	3.55
Asma Yusuf	3.30
Ruqayyah Hasan	3.90
Aaliyah Karim	3.40
Nusaybah Omar	3.75

Figure 120: Table Displayed

2.4 Display all student names that have more than a 3.5 CGPA

```
SELECT Name FROM STUDENT WHERE CGPA > 3.5;
```

Name
Aisha Khan
Fatima Noor
Zainab Ahmed
Maryam Siddiqui
Sumayya Ali
Ruqayyah Hasan
Nusaybah Omar

Figure 121: Table Displayed

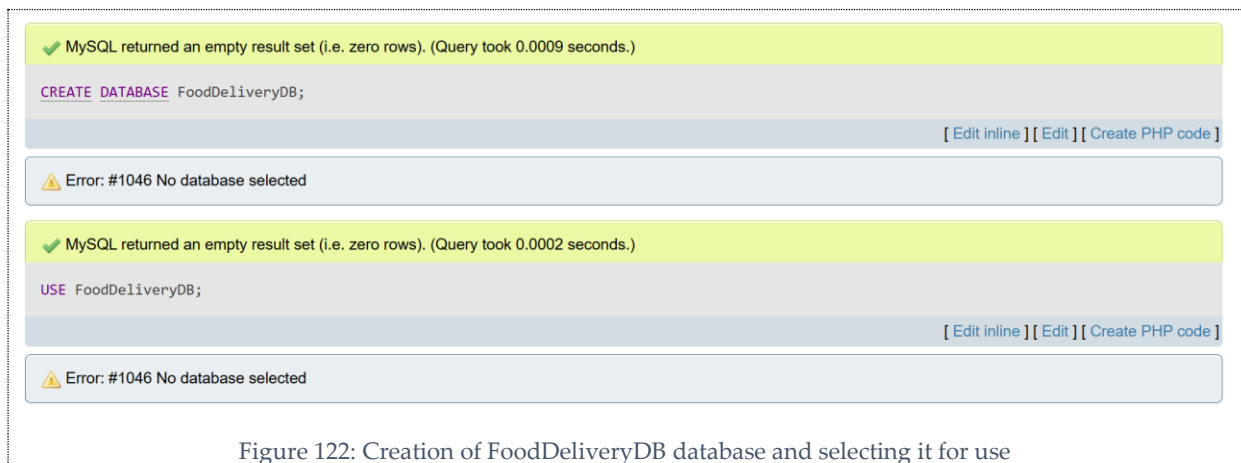
LAB NO 11 : Practical Test(Create Database and Table, Alter, Drop, Insert, Update, Delete, Where, Order By and Other Restricting Data Constraints

SCENARIO NO 1

ONLINE FOOD DELIVERY

1. Create a database

```
CREATE DATABASE FoodDeliveryDB;
USE FoodDeliveryDB;
```



2. Create a table

```
CREATE TABLE Orders (
    OrderID INT PRIMARY KEY,
    CustomerName VARCHAR(100),
    FoodItem VARCHAR(100),
    Quantity INT CHECK (Quantity > 0),
    OrderDate DATE,
    DeliveryStatus VARCHAR(50) DEFAULT 'Pending'
```

```
);
```

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0004 seconds.)

```
CREATE TABLE Orders ( OrderID INT PRIMARY KEY, CustomerName VARCHAR(100), FoodItem VARCHAR(100), Quantity INT
CHECK (Quantity > 0), OrderDate DATE, DeliveryStatus VARCHAR(50) DEFAULT 'Pending' );
```

[Edit inline] [Edit] [Create PHP code]

Figure 123: Table Orders created with fields including OrderDate, Quantity, and DeliveryStatus

3. Show all databases and show tables inside FoodDeliveryDB

```
SHOW DATABASES;
```

```
USE FoodDeliveryDB;
```

```
SHOW TABLES;
```

Your SQL query has been executed successfully.

```
SHOW DATABASES;
```

☐ Profiling [Edit inline] [Edit] [Create PHP code] [Refresh]

Database

fooddeliverydb

hospitaldb

information_schema

mysql

performance_schema

phpmyadmin

student_info

test

tuf_db

Figure 124: Displaying all databases and tables within FoodDeliveryDB

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0002 seconds.)

```
USE FoodDeliveryDB;
```

[Edit inline] [Edit] [Create PHP code]

⚠ Current selection does not contain a unique column. Grid edit, checkbox, Edit, Copy and Delete features are not available. ⓘ

Your SQL query has been executed successfully.

```
SHOW TABLES;
```

☐ Profiling [Edit inline] [Edit] [Create PHP code] [Refresh]

4. Add a new column

```
ALTER TABLE Orders ADD TotalPrice DECIMAL(10, 2);
```

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0004 seconds.)

```
ALTER TABLE Orders ADD TotalPrice DECIMAL(10, 2);
```

[\[Edit inline \]](#) [\[Edit \]](#) [\[Create PHP code \]](#)

Figure 125: Adding TotalPrice column to Orders table

5. Insert ten food order records

```
INSERT INTO Orders (OrderID, CustomerName, FoodItem, Quantity,
OrderDate, DeliveryStatus, TotalPrice) VALUES
(201, 'Ali', 'Burger', 2, '2025-05-10', 'Pending', 500.00),
(202, 'Sara', 'Pizza', 1, '2025-05-11', 'Pending', 800.00),
(203, 'Zain', 'Biryani', 3, '2025-05-11', 'Pending', 750.00),
(204, 'Areeba', 'Pasta', 2, '2025-05-12', 'Pending', 600.00),
(205, 'Hassan', 'Sandwich', 1, '2025-05-12', 'Pending', 300.00),
(206, 'Tariq', 'Nuggets', 4, '2025-05-12', 'Pending', 900.00),
(207, 'Noor', 'Burger', 2, '2025-05-13', 'Pending', 500.00),
(208, 'Fahad', 'Fries', 1, '2025-05-13', 'Pending', 200.00),
(209, 'Lubna', 'Pizza', 2, '2025-05-13', 'Pending', 1600.00),
(210, 'Asad', 'Biryani', 5, '2025-05-13', 'Pending', 1250.00);
```

✓ 10 rows inserted. (Query took 0.0005 seconds.)

```
INSERT INTO Orders (OrderID, CustomerName, FoodItem, Quantity, OrderDate, DeliveryStatus, TotalPrice) VALUES
(201, 'Ali', 'Burger', 2, '2025-05-10', 'Pending', 500.00), (202, 'Sara', 'Pizza', 1, '2025-05-11', 'Pending',
800.00), (203, 'Zain', 'Biryani', 3, '2025-05-11', 'Pending', 750.00), (204, 'Areeba', 'Pasta', 2, '2025-05-
12', 'Pending', 600.00), (205, 'Hassan', 'Sandwich', 1, '2025-05-12', 'Pending', 300.00), (206, 'Tariq',
'Nuggets', 4, '2025-05-12', 'Pending', 900.00), (207, 'Noor', 'Burger', 2, '2025-05-13', 'Pending', 500.00),
(208, 'Fahad', 'Fries', 1, '2025-05-13', 'Pending', 200.00), (209, 'Lubna', 'Pizza', 2, '2025-05-13',
```

[\[Edit inline \]](#) [\[Edit \]](#) [\[Create PHP code \]](#)

Figure 126: Inserting 10 food order records into the Orders table

6. Update the delivery status

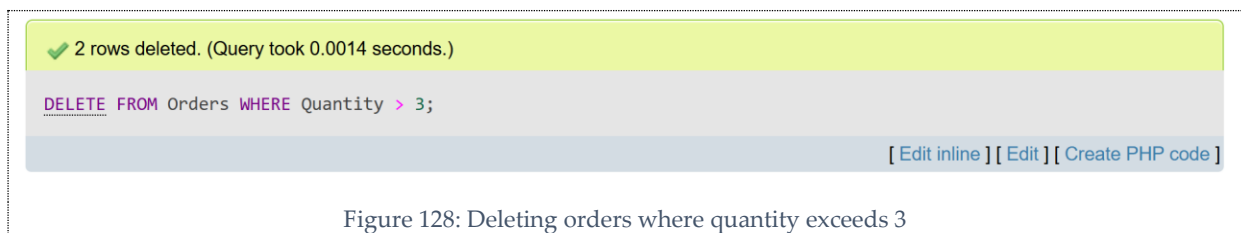
```
UPDATE Orders
```

```
SET DeliveryStatus = 'Delivered'
WHERE OrderID = 202;
```



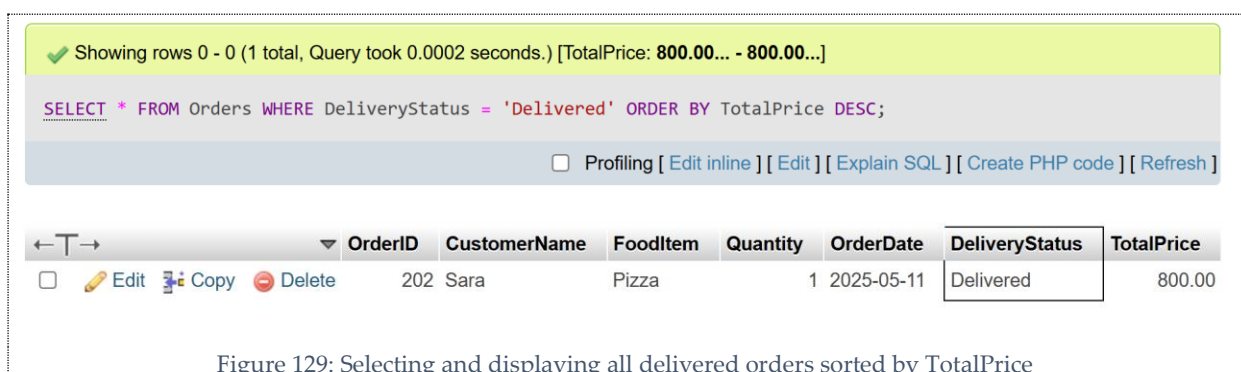
7. Delete any orders where the quantity is greater than 3

```
DELETE FROM Orders
WHERE Quantity > 3;
```



8. Select and display all delivered orders

```
SELECT *
FROM Orders
WHERE DeliveryStatus = 'Delivered'
ORDER BY TotalPrice DESC;
```



9. Drop the Orders table

```
DROP TABLE Orders;
```

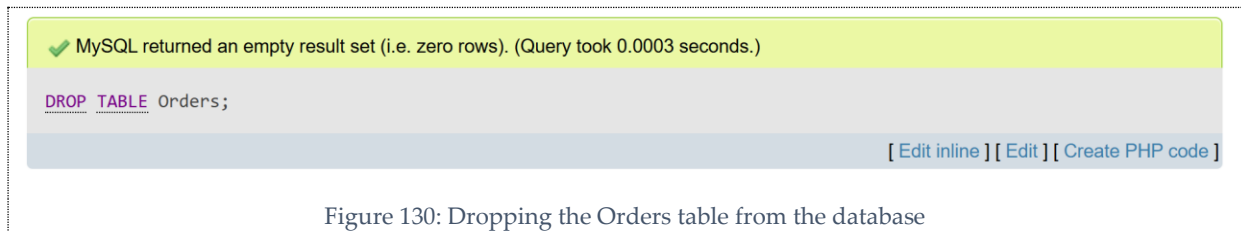


Figure 130: Dropping the Orders table from the database

SCENARIO NO 2

HOSPITAL MANAGEMENT

1. Create a database

2.

```
CREATE DATABASE HospitalDB;
```

```
USE HospitalDB;
```

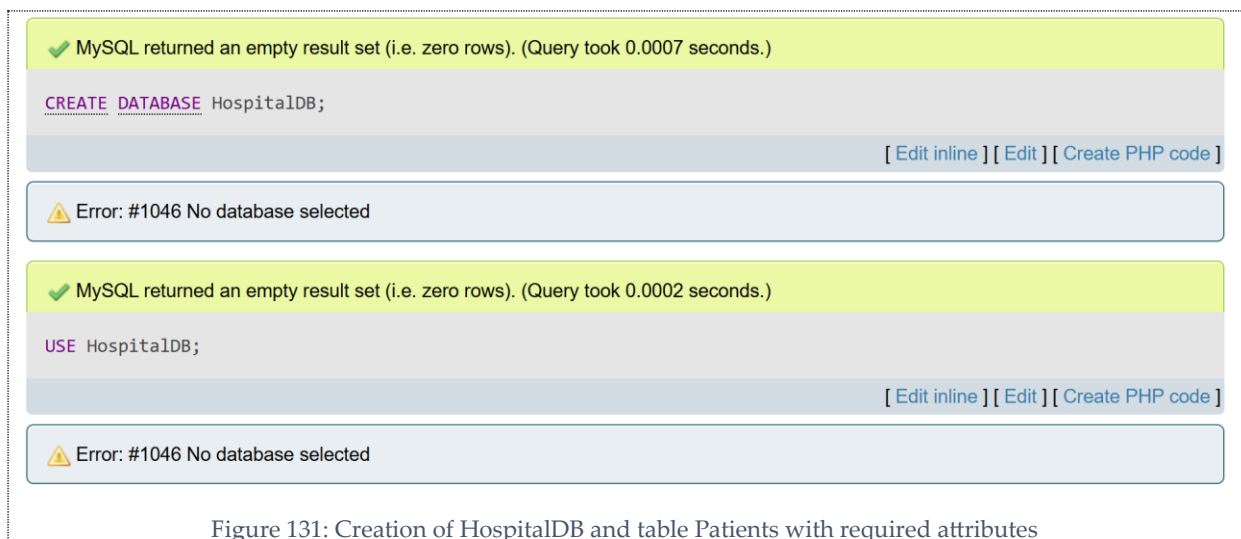


Figure 131: Creation of HospitalDB and table Patients with required attributes

3. Create a table

```
CREATE TABLE Patients (  
    PatientID INT PRIMARY KEY,
```

```

FullName VARCHAR(100),
Age INT CHECK (Age >= 0),
Gender VARCHAR(10),
AdmissionDate DATE NOT NULL
);

```

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0003 seconds.)

```

CREATE TABLE Patients ( PatientID INT PRIMARY KEY, FullName VARCHAR(100), Age INT CHECK (Age >= 0), Gender
VARCHAR(10), AdmissionDate DATE NOT NULL );

```

[\[Edit inline \]](#) [\[Edit \]](#) [\[Create PHP code \]](#)

4. Show all databases and all tables

```
SHOW DATABASES;
```

```
USE HospitalDB;
```

```
SHOW TABLES;
```

Your SQL query has been executed successfully.

```
SHOW DATABASES;
```

☐ Profiling [\[Edit inline \]](#) [\[Edit \]](#) [\[Create PHP code \]](#) [\[Refresh \]](#)

Database

hospitaldb

information_schema

mysql

performance_schema

phpmyadmin

student_info

test

tuf_db

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0002 seconds.)

```
USE HospitalDB;
```

[\[Edit inline \]](#) [\[Edit \]](#) [\[Create PHP code \]](#)

⚠ Current selection does not contain a unique column. Grid edit, checkbox, Edit, Copy and Delete features are not available. [🔗](#)

Your SQL query has been executed successfully.

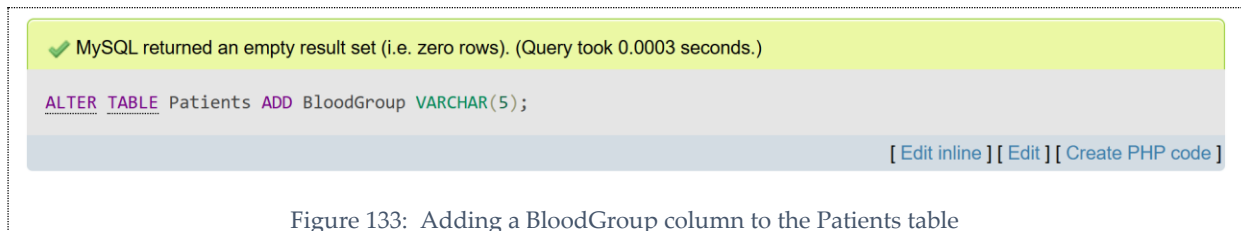
```
SHOW TABLES;
```

☐ Profiling [\[Edit inline \]](#) [\[Edit \]](#) [\[Create PHP code \]](#) [\[Refresh \]](#)

Figure 132: Showing databases and verifying tables inside HospitalDB

5. Add a new column

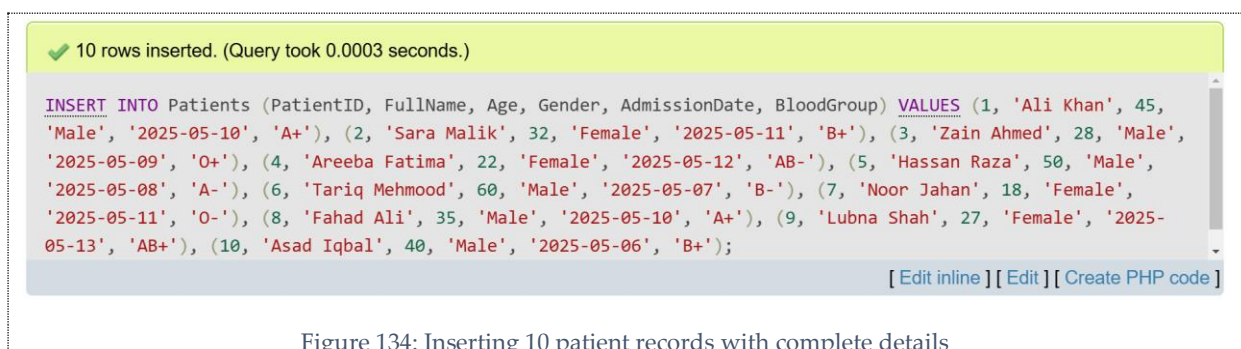
```
ALTER TABLE Patients ADD BloodGroup VARCHAR(5);
```



6. Insert ten patient records

```
INSERT INTO Patients (PatientID, FullName, Age, Gender, AdmissionDate, BloodGroup) VALUES
```

```
(1, 'Ali Khan', 45, 'Male', '2025-05-10', 'A+'),
(2, 'Sara Malik', 32, 'Female', '2025-05-11', 'B+'),
(3, 'Zain Ahmed', 28, 'Male', '2025-05-09', 'O+'),
(4, 'Areeba Fatima', 22, 'Female', '2025-05-12', 'AB-'),
(5, 'Hassan Raza', 50, 'Male', '2025-05-08', 'A-'),
(6, 'Tariq Mehmood', 60, 'Male', '2025-05-07', 'B-'),
(7, 'Noor Jahan', 18, 'Female', '2025-05-11', 'O-'),
(8, 'Fahad Ali', 35, 'Male', '2025-05-10', 'A+'),
(9, 'Lubna Shah', 27, 'Female', '2025-05-13', 'AB+'),
(10, 'Asad Iqbal', 40, 'Male', '2025-05-06', 'B+');
```



7. Update the age of the patient

```
UPDATE Patients  
SET Age = 46  
WHERE PatientID = 1;
```

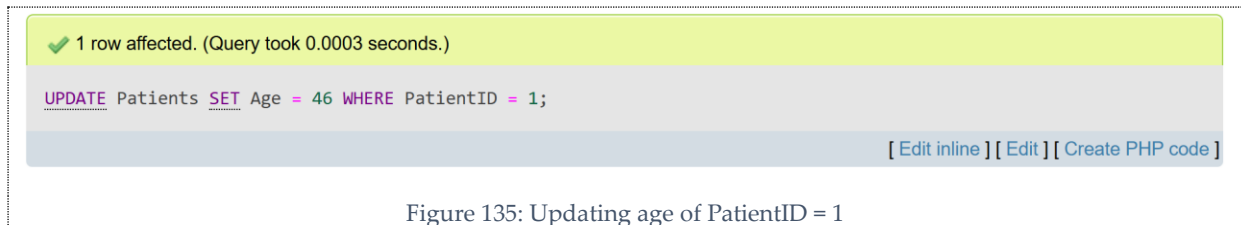


Figure 135: Updating age of PatientID = 1

8. Delete the record(s) of patients whose gender is 'Female'

```
DELETE FROM Patients  
WHERE Gender = 'Female';
```

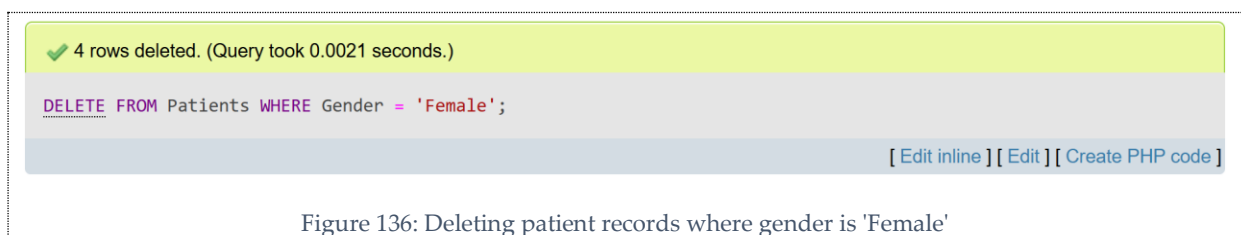
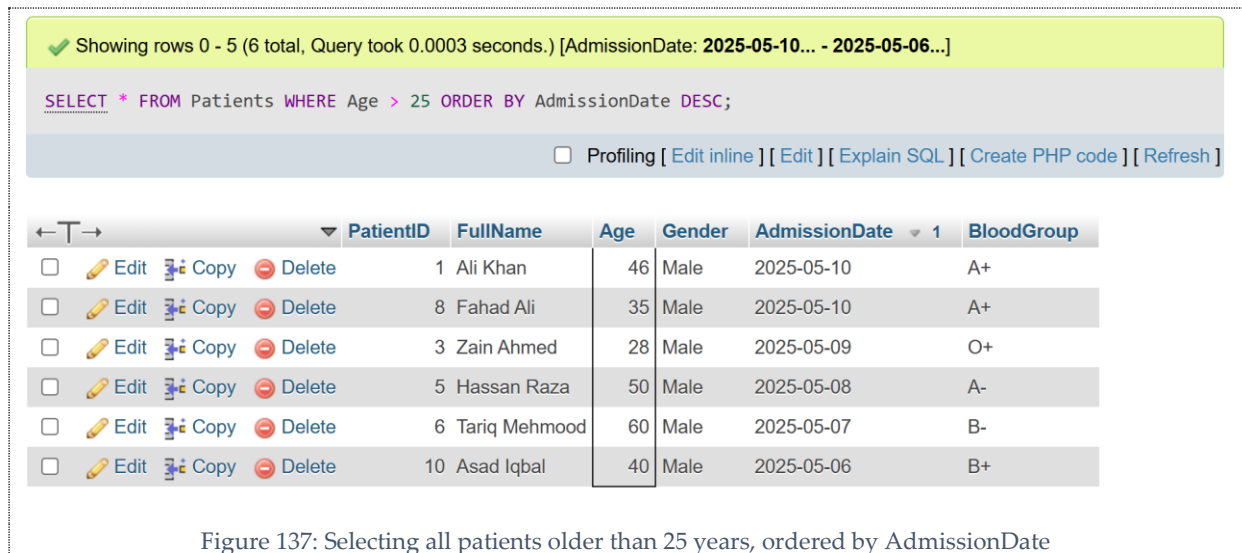


Figure 136: Deleting patient records where gender is 'Female'

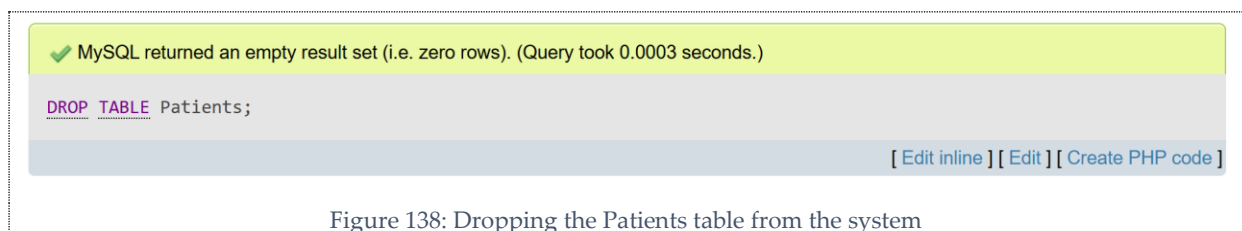
9. Select and display all patients older than 25 years

```
SELECT *  
FROM Patients  
WHERE Age > 25  
ORDER BY AdmissionDate DESC;
```



10. Drop the entire Patients table

DROP TABLE Patients;



SCENARIO NO 3

UNIVERSITY COURSE SYSTEM

1. Create a database

```
CREATE DATABASE UniversityDB;

USE UniversityDB;
```

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0004 seconds.)

```
CREATE DATABASE UniversityDB;
```

[\[Edit inline \]](#) [\[Edit \]](#) [\[Create PHP code \]](#)

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0001 seconds.)

```
USE UniversityDB;
```

[\[Edit inline \]](#) [\[Edit \]](#) [\[Create PHP code \]](#)

2. Create a table

```
CREATE TABLE Courses (
    CourseID INT PRIMARY KEY,
    CourseName VARCHAR(100),
    Instructor VARCHAR(100),
    Credits INT CHECK (Credits BETWEEN 1 AND 6)
);
```

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0004 seconds.)

```
CREATE TABLE Courses ( CourseID INT PRIMARY KEY, CourseName VARCHAR(100), Instructor VARCHAR(100), Credits INT
CHECK (Credits BETWEEN 1 AND 6) );
```

[\[Edit inline \]](#) [\[Edit \]](#) [\[Create PHP code \]](#)

3. Show all databases and show tables

```
SHOW DATABASES;

USE UniversityDB;

SHOW TABLES;
```

Your SQL query has been executed successfully.

[SHOW DATABASES;](#)

☐ Profiling [\[Edit inline \]](#) [\[Edit \]](#) [\[Create PHP code \]](#) [\[Refresh \]](#)

Database

hospitaldb

information_schema

mysql

performance_schema

phpmyadmin

student_info

test

tuf_db

universitydb

4. Add a new column

```
ALTER TABLE Courses ADD Department VARCHAR(100);
```

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0003 seconds.)

[ALTER TABLE](#) Courses [ADD](#) Department [VARCHAR\(100\);](#)

[\[Edit inline \]](#) [\[Edit \]](#) [\[Create PHP code \]](#)

5. Insert ten course records

```
INSERT INTO Courses (CourseID, CourseName, Instructor, Credits,
Department) VALUES
```

```
(101, 'Data Structures', 'Dr. Smith', 3, 'Computer Science'),
```

```
(102, 'Calculus', 'Prof. Khan', 4, 'Mathematics'),
```

```
(103, 'Operating Systems', 'Dr. Lee', 3, 'Computer Science'),
```

```
(104, 'Microeconomics', 'Dr. Fatima', 2, 'Economics'),
```

```
(105, 'Database Systems', 'Dr. Alice', 3, 'Computer Science'),
```

```
(106, 'Linear Algebra', 'Prof. Ahmed', 4, 'Mathematics'),
```

```
(107, 'Networking', 'Dr. Imran', 3, 'Computer Science'),
```

```
(108, 'Statistics', 'Dr. Sana', 2, 'Mathematics'),
```

```
(109, 'Artificial Intelligence', 'Dr. Yasir', 5, 'Computer Science'),
```

```
(110, 'Psychology 101', 'Dr. Maria', 3, 'Psychology');
```

✓ 10 rows inserted. (Query took 0.0004 seconds.)

```
INSERT INTO Courses (CourseID, CourseName, Instructor, Credits, Department) VALUES (101, 'Data Structures', 'Dr. Smith', 3, 'Computer Science'), (102, 'Calculus', 'Prof. Khan', 4, 'Mathematics'), (103, 'Operating Systems', 'Dr. Lee', 3, 'Computer Science'), (104, 'Microeconomics', 'Dr. Fatima', 2, 'Economics'), (105, 'Database Systems', 'Dr. Alice', 3, 'Computer Science'), (106, 'Linear Algebra', 'Prof. Ahmed', 4, 'Mathematics'), (107, 'Networking', 'Dr. Imran', 3, 'Computer Science'), (108, 'Statistics', 'Dr. Sana', 2, 'Mathematics'), (109, 'Artificial Intelligence', 'Dr. Yasir', 5, 'Computer Science'), (110, 'Psychology 101',
```

[\[Edit inline \]](#) [\[Edit \]](#) [\[Create PHP code \]](#)

6. Update the instructor's name for the course with CourseID = 101

UPDATE Courses

SET Instructor = 'Dr. Alice Allen'

WHERE CourseID = 101;

✓ 1 row affected. (Query took 0.0003 seconds.)

```
UPDATE Courses SET Instructor = 'Dr. Alice Allen' WHERE CourseID = 101;
```

[\[Edit inline \]](#) [\[Edit \]](#) [\[Create PHP code \]](#)

7. Delete the course record where the credits are equal to 4

DELETE FROM Courses

WHERE Credits = 4;

✓ 2 rows deleted. (Query took 0.0009 seconds.)

```
DELETE FROM Courses WHERE Credits = 4;
```

[\[Edit inline \]](#) [\[Edit \]](#) [\[Create PHP code \]](#)

8. Select and display all courses in the 'Computer Science' department, ordered by course name

SELECT *

FROM Courses

WHERE Department = 'Computer Science'

ORDER BY CourseName ASC;

✓ Showing rows 0 - 4 (5 total, Query took 0.0003 seconds.) [CourseName: ARTIFICIAL INTELLIGENCE... - OPERATING SYSTEMS...]

```
SELECT * FROM Courses WHERE Department = 'Computer Science' ORDER BY CourseName ASC;
```

☐ Profiling [\[Edit inline \]](#) [\[Edit \]](#) [\[Explain SQL \]](#) [\[Create PHP code \]](#) [\[Refresh \]](#)

← T →				▼ CourseID	CourseName ▲ 1	Instructor	Credits	Department
☐				109	Artificial Intelligence	Dr. Yasir	5	Computer Science
☐				101	Data Structures	Dr. Alice Allen	3	Computer Science
☐				105	Database Systems	Dr. Alice	3	Computer Science
☐				107	Networking	Dr. Imran	3	Computer Science
☐				103	Operating Systems	Dr. Lee	3	Computer Science

9. Drop the Courses table

DROP TABLE Courses;

✓ MySQL returned an empty result set (i.e. zero rows). (Query took 0.0003 seconds.)

DROP TABLE Courses;

[[Edit inline](#)] [[Edit](#)] [[Create PHP code](#)]

LAB 12: JOIN (CROSS, INNER AND OUTER JOIN (LEFT, RIGHT, FULL))

1. Performing Joins on Employee and Department Table

1.1 Create Employee Table

```
CREATE TABLE EMPLOYEE(
FIRST_NAME VARCHAR(30),
LAST_NAME VARCHAR(30),
SALARY INT,
DEPARTMENT_ID INT
);
```

TABLE STRUCTURE									
#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐ 1	FIRST_NAME	varchar(30)	latin1_swedish_ci		Yes	NULL			Change Drop More
☐ 2	LAST_NAME	varchar(30)	latin1_swedish_ci		Yes	NULL			Change Drop More
☐ 3	SALARY	int(11)			Yes	NULL			Change Drop More
☐ 4	DEPARTMENT_ID	int(11)			Yes	NULL			Change Drop More

Figure 139: Employee Table Created

1.2 Inserting Data into Employee

```
INSERT INTO EMPLOYEE
VALUES
('Aisha', 'Fatimi', 1500, 50),
('Layla', 'Siddiqui', 2700, 50),
('Zainab', 'Hussain', 3800, 90),
('Mariam', 'Khan', 4200, 90);
```

FIRST_NAME	LAST_NAME	SALARY	DEPARTMENT_ID
Aisha	Fatimi	1500	50
Layla	Siddiqui	2700	50
Zainab	Hussain	3800	90
Mariam	Khan	4200	90

Figure 140: Values Inserted

1.3 Create Department

```
CREATE TABLE DEPARTMENT(
DEPARTMENT_ID INT,
DEPARTMENT_NAME VARCHAR(40),
REGION_ID INT
);
```

TABLE STRUCTURE									
#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐ 1	DEPARTMENT_ID	int(11)			Yes	NULL			Change Drop More
☐ 2	DEPARTMENT_NAME	varchar(40)	latin1_swedish_ci		Yes	NULL			Change Drop More
☐ 3	REGION_ID	int(11)			Yes	NULL			Change Drop More

Figure 141: Table Structure

1.4 Inserting Values

```
INSERT INTO department (DEPARTMENT_ID, DEPARTMENT_NAME, REGION_ID)
VALUES
```

```
(1, 'Sales', 101),
(2, 'Marketing', 102),
(3, 'Finance', 103),
(4, 'HR', 104),
(5, 'IT', 105),
(6, 'Operations', 106),
```

```
(7, 'Legal', 107),
(8, 'Research', 108),
(9, 'Customer Service', 109),
(10, 'Logistics', 110);
```

DEPARTMENT_ID	DEPARTMENT_NAME	REGION_ID
1	Sales	101
2	Marketing	102
3	Finance	103
4	HR	104
5	IT	105
6	Operations	106
7	Legal	107
8	Research	108
9	Customer Service	109
10	Logistics	110

Figure 142: Data Inserted

1.5 Cross Join Query

```
SELECT * from employee cross join department;
```

FIRST_NAME	LAST_NAME	SALARY	DEPARTMENT_ID	DEPARTMENT_ID	DEPARTMENT_NAME	REGION_ID
Aisha	Fatimi	1500	50	1	Sales	101
Layla	Siddiqui	2700	50	1	Sales	101
Zainab	Hussain	3800	90	1	Sales	101
Mariam	Khan	4200	90	1	Sales	101
Aisha	Fatimi	1500	50	2	Marketing	102
Layla	Siddiqui	2700	50	2	Marketing	102
Zainab	Hussain	3800	90	2	Marketing	102
Mariam	Khan	4200	90	2	Marketing	102
Aisha	Fatimi	1500	50	3	Finance	103
Layla	Siddiqui	2700	50	3	Finance	103
Zainab	Hussain	3800	90	3	Finance	103
Mariam	Khan	4200	90	3	Finance	103
Aisha	Fatimi	1500	50	4	HR	104
Layla	Siddiqui	2700	50	4	HR	104
Zainab	Hussain	3800	90	4	HR	104
Mariam	Khan	4200	90	4	HR	104
Aisha	Fatimi	1500	50	5	IT	105
Layla	Siddiqui	2700	50	5	IT	105
Zainab	Hussain	3800	90	5	IT	105
Mariam	Khan	4200	90	5	IT	105
Aisha	Fatimi	1500	50	6	Operations	106
Layla	Siddiqui	2700	50	6	Operations	106
Zainab	Hussain	3800	90	6	Operations	106
Mariam	Khan	4200	90	6	Operations	106
Aisha	Fatimi	1500	50	7	Legal	107

Figure 143: Cross Join of "Employee" and "Department"

1.6 Inner Join Query

```
SELECT EMPLOYEE.FIRST_NAME, DEPARTMENT.DEPARTMENT_NAME
FROM EMPLOYEE INNER JOIN DEPARTMENT
ON EMPLOYEE.DEPARTMENT_ID = DEPARTMENT.DEPARTMENT_ID;
```

FIRST_NAME	DEPARTMENT_NAME
------------	-----------------

Figure 144: No Result

1.7 Right Join Query

```
SELECT EMPLOYEE.FIRST_NAME, DEPARTMENT.DEPARTMENT_NAME
FROM EMPLOYEE RIGHT JOIN DEPARTMENT
ON EMPLOYEE.DEPARTMENT_ID = DEPARTMENT.DEPARTMENT_ID;
```

FIRST_NAME	DEPARTMENT_NAME
NULL	Sales
NULL	Marketing
NULL	Finance
NULL	HR
NULL	IT
NULL	Operations
NULL	Legal
NULL	Research
NULL	Customer Service
NULL	Logistics

Figure 145: Table Displayed

1.8 Left Join Query

```
SELECT EMPLOYEE.FIRST_NAME, DEPARTMENT.DEPARTMENT_NAME
```

```
FROM EMPLOYEE LEFT JOIN DEPARTMENT
ON EMPLOYEE.DEPARTMENT_ID = DEPARTMENT.DEPARTMENT_ID;
```

FIRST_NAME	DEPARTMENT_NAME
Aisha	NULL
Layla	NULL
Zainab	NULL
Mariam	NULL

Figure 146: Table Displayed

2. Performing Joins on Employee, Project and Client Table

2.1 Inner Join Query

```
SELECT * FROM EMPLOYEE INNER JOIN PROJECT
ON EMPLOYEE.EMPID = PROJECT.EMPID;
```

EMPID	EMPNAME	DEPARTMENT	PROJECTID	PROJECTNAME	EMPID
1	Ali Raza	IT	101	E-Commerce App	1
2	Zara Khan	HR	102	HR Portal	2
3	Ahmed Noor	Finance	103	Budget Tracker	3

Figure 147: Table Displayed

2.2 Left Join Query

EMPID	EMPNAME	DEPARTMENT	PROJECTID	PROJECTNAME	EMPID
1	Ali Raza	IT	101	E-Commerce App	1
2	Zara Khan	HR	102	HR Portal	2
3	Ahmed Noor	Finance	103	Budget Tracker	3
4	Fatima Tariq	Marketing	NULL	NULL	NULL

Figure 148: Table Displayed

2.3 Inner Join Query

```
SELECT * FROM PROJECT INNER JOIN CLIENT ON PROJECT.EMPID =
CLIENT.EMPID;
```

PROJECTID	PROJECTNAME	EMPID	CLIENTID	CLIENTNAME	EMPID
101	E-Commerce App	1	201	Telenor	1
102	HR Portal	2	202	Jazz	2

Figure 149: Table Displayed

2.4 Right Join Query

```
SELECT * FROM PROJECT RIGHT JOIN CLIENT
ON PROJECT.EMPID = CLIENT.EMPID;
```

PROJECTID	PROJECTNAME	EMPID	CLIENTID	CLIENTNAME	EMPID
101	E-Commerce App	1	201	Telenor	1
102	HR Portal	2	202	Jazz	2
NULL	NULL	NULL	203	PTCL	NULL
NULL	NULL	NULL	204	HBL	4

Figure 150: Table Displayed

2.5 Left Join Query

```
SELECT * FROM PROJECT LEFT JOIN CLIENT
ON PROJECT.EMPID = CLIENT.EMPID;
```

PROJECTID	PROJECTNAME	EMPID	CLIENTID	CLIENTNAME	EMPID
101	E-Commerce App	1	201	Telenor	1
102	HR Portal	2	202	Jazz	2
103	Budget Tracker	3	NULL	NULL	NULL
104	Social Media Campaign	NULL	NULL	NULL	NULL

Figure 151: Table Displayed

LAB NO 13: SELF-JOIN, MULTI JOIN, AND UNION

1. Creating Tables

```
CREATE TABLE COACH (
    ID INT PRIMARY KEY,
    NAME VARCHAR(100)
);
```

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	ID	int(11)			No	None			Change Drop More
2	NAME	varchar(100)	latin1_swedish_ci		Yes	NULL			Change Drop More

Figure 152: Table Coach Created

```
CREATE TABLE TEAM (
    ID INT PRIMARY KEY,
    NAME VARCHAR(100),
    COACH_ID INT,
    FOREIGN KEY (COACH_ID) REFERENCES COACH(ID)
);
```

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
1	ID	int(11)			No	None			Change Drop More
2	NAME	varchar(100)	latin1_swedish_ci		Yes	NULL			Change Drop More
3	COACH_ID	int(11)			Yes	NULL			Change Drop More

Figure 153: Table Team Created

```
CREATE TABLE PLAYER (
    ID INT PRIMARY KEY,
    NAME VARCHAR(100),
    TEAM_ID INT,
    FOREIGN KEY (TEAM_ID) REFERENCES TEAM(ID)
);
```

TABLE STRUCTURE

RELATION VIEW

#	Name	Type	Collation	Attributes	Null	Default	Comments	Extra	Action
☐ 1	ID	int(11)			No	None			Change Drop More
☐ 2	NAME	varchar(100)	latin1_swedish_ci		Yes	NULL			Change Drop More
☐ 3	TEAM_ID	int(11)			Yes	NULL			Change Drop More

Figure 154: Table Created

2. Perform Join Operations

2.1 Cross Join

```
SELECT* FROM COACH CROSS JOIN PLAYER
```

ID	NAME	ID	NAME	TEAM_ID
1	Imran Malik	1	Babar Azam	1
2	Sana Javed	1	Babar Azam	1
1	Imran Malik	2	Shaheen Afridi	1
2	Sana Javed	2	Shaheen Afridi	1
1	Imran Malik	3	Mohammad Rizwan	2
2	Sana Javed	3	Mohammad Rizwan	2
1	Imran Malik	4	Shadab Khan	3
2	Sana Javed	4	Shadab Khan	3

Figure 155: Table Displayed

2.2 Inner Join Query

```
SELECT* FROM COACH INNER JOIN TEAM
ON COACH.ID = TEAM.COACH_ID
```

ID	NAME	ID	NAME	COACH_ID
1	Imran Malik	1	Lahore Qalandars	1
2	Sana Javed	2	Karachi Kings	2

Figure 156: Table Displayed

2.3 Left Join Query

```
SELECT* FROM COACH LEFT JOIN TEAM
ON COACH.ID = TEAM.COACH_ID
```

ID	NAME	ID	NAME	COACH_ID
1	Imran Malik	1	Lahore Qalandars	1
2	Sana Javed	2	Karachi Kings	2

Figure 157: Table Displayed

2.4 Right Join Query

```
SELECT* FROM COACH RIGHT JOIN TEAM
ON COACH.ID = TEAM.COACH_ID
```

ID	NAME	ID	NAME	COACH_ID
1	Imran Malik	1	Lahore Qalandars	1
2	Sana Javed	2	Karachi Kings	2
NULL	NULL	3	Multan Sultans	NULL

Figure 158: Table Displayed

2.5 Self Join Query

SELECT

C1.ID AS COACH_ID_1, C1.NAME AS COACH_NAME_1,

C2.ID AS COACH_ID_2, C2.NAME AS COACH_NAME_2

FROM COACH C1, COACH C2

WHERE

C1.ID > C2.ID;

COACH_ID_1	COACH_NAME_1	COACH_ID_2	COACH_NAME_2
2	Sana Javed	1	Imran Malik

Figure 159: Table Displayed

2.6 Multiple Join Query

SELECT

P.NAME AS PLAYER_NAME,

T.NAME AS TEAM_NAME,

C.NAME AS COACH_NAME

FROM PLAYER P

JOIN

TEAM T ON P.TEAM_ID = T.ID

JOIN

COACH C ON T.COACH_ID = C.ID;

PLAYER_NAME	TEAM_NAME	COACH_NAME
Babar Azam	Lahore Qalandars	Imran Malik
Shaheen Afridi	Lahore Qalandars	Imran Malik
Mohammad Rizwan	Karachi Kings	Sana Javed

Figure 160: Table Displayed

LAB NO 14: SQL FUNCTIONS (MATH AND STRING)

```
CREATE DATABASE Book;
```

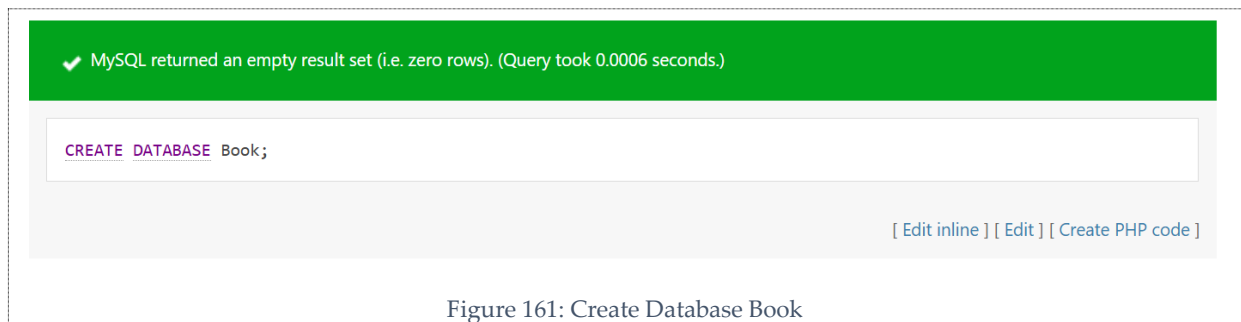


Figure 161: Create Database Book

```
USE DATABASE Book;
```

```
CREATE TABLE BookTable (
```

```
    ISBN VARCHAR(10),
```

```
    Title VARCHAR(100),
```

```
    Quantity INT,
```

```
    Price DECIMAL(10, 2),
```

```
    PurchaseDate DATE
```

```
);
```

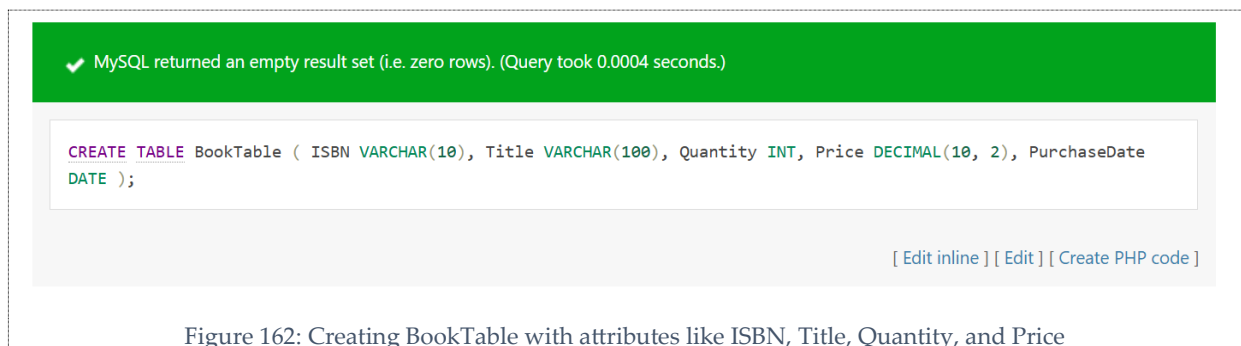


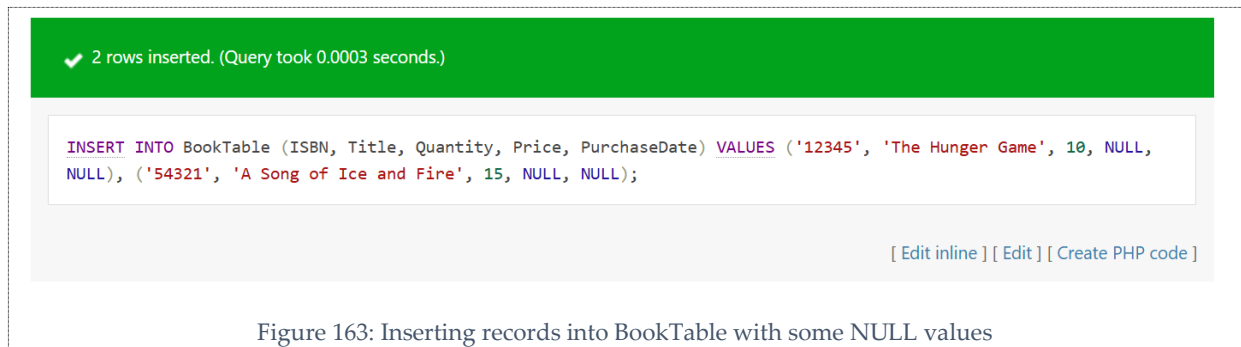
Figure 162: Creating BookTable with attributes like ISBN, Title, Quantity, and Price

```
INSERT INTO BookTable (ISBN, Title, Quantity, Price, PurchaseDate)
```

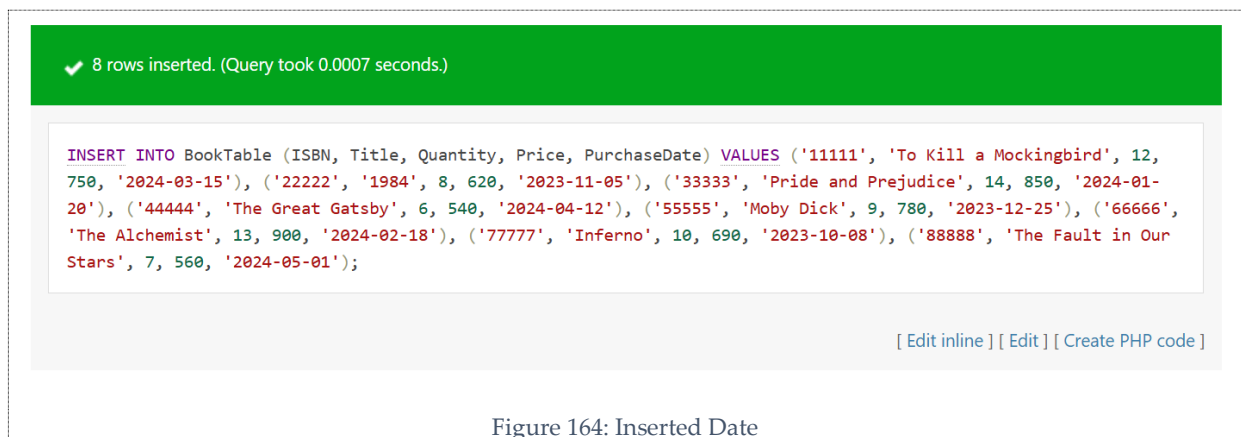
```
VALUES
```

```
('12345', 'The Hunger Game', 10, NULL, NULL),
```

```
('54321', 'A Song of Ice and Fire', 15, NULL, NULL);
```



```
INSERT INTO BookTable (ISBN, Title, Quantity, Price, PurchaseDate)
VALUES
('11111', 'To Kill a Mockingbird', 12, 750, '2024-03-15'),
('22222', '1984', 8, 620, '2023-11-05'),
('33333', 'Pride and Prejudice', 14, 850, '2024-01-20'),
('44444', 'The Great Gatsby', 6, 540, '2024-04-12'),
('55555', 'Moby Dick', 9, 780, '2023-12-25'),
('66666', 'The Alchemist', 13, 900, '2024-02-18'),
('77777', 'Inferno', 10, 690, '2023-10-08'),
('88888', 'The Fault in Our Stars', 7, 560, '2024-05-01');
```



```
UPDATE BookTable
SET Price = 670, PurchaseDate = '2023-09-10'
WHERE Title = 'The Hunger Game';
```

```
UPDATE BookTable
```

```
SET Price = 940, PurchaseDate = '2024-06-01'
WHERE Title = 'A Song of Ice and Fire';
```



✓ 1 row affected. (Query took 0.0004 seconds.)

```
UPDATE BookTable SET Price = 670, PurchaseDate = '2023-09-10' WHERE Title = 'The Hunger Game';
```

[Edit inline] [Edit] [Create PHP code]

✓ 1 row affected. (Query took 0.0004 seconds.)

```
UPDATE BookTable SET Price = 940, PurchaseDate = '2024-06-01' WHERE Title = 'A Song of Ice and Fire';
```

[Edit inline] [Edit] [Create PHP code]

Figure 165: Updating Price and PurchaseDate for specific books

ISBN	Title	Quantity	Price	PurchaseDate
12345	The Hunger Game	10	670.00	2023-09-10
54321	A Song of Ice and Fire	15	940.00	2024-06-01
11111	To Kill a Mockingbird	12	750.00	2024-03-15
22222	1984	8	620.00	2023-11-05
33333	Pride and Prejudice	14	850.00	2024-01-20
44444	The Great Gatsby	6	540.00	2024-04-12
55555	Moby Dick	9	780.00	2023-12-25
66666	The Alchemist	13	900.00	2024-02-18
77777	Inferno	10	690.00	2023-10-08
88888	The Fault in Our Stars	7	560.00	2024-05-01

Figure 166: New Data

```
SELECT ISBN, Title, (Quantity + 3) * 2 AS ModifiedQuantity
FROM BookTable;
```

✓ Showing rows 0 - 9 (10 total, Query took 0.0003 seconds.)

```
SELECT ISBN, Title, (Quantity + 3) * 2 AS ModifiedQuantity FROM BookTable;
```

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

Figure 167: Retrieving calculated field ModifiedQuantity using arithmetic in SELECT

ISBN	Title	ModifiedQuantity
12345	The Hunger Game	26
54321	A Song of Ice and Fire	36
11111	To Kill a Mockingbird	30
22222	1984	22
33333	Pride and Prejudice	34
44444	The Great Gatsby	18
55555	Moby Dick	24
66666	The Alchemist	32
77777	Inferno	26
88888	The Fault in Our Stars	20

Figure 168: Result Table

```
SELECT ISBN, Title, (Quantity + 3) * 2 FROM BookTable;
```

ISBN	Title	(Quantity + 3) * 2
12345	The Hunger Game	26
54321	A Song of Ice and Fire	36
11111	To Kill a Mockingbird	30
22222	1984	22
33333	Pride and Prejudice	34
44444	The Great Gatsby	18
55555	Moby Dick	24
66666	The Alchemist	32
77777	Inferno	26
88888	The Fault in Our Stars	20

Figure 169: Result Table

```
SELECT ISBN, Quantity -13, ABS(QUANTITY -13)
FROM booktable;
```

✓ Showing rows 0 - 9 (10 total, Query took 0.0003 seconds.)

```
SELECT ISBN, Quantity -13, ABS(QUANTITY -13) FROM booktable;
```

☐ Profiling [[Edit inline](#)] [[Edit](#)] [[Explain SQL](#)] [[Create PHP code](#)] [[Refresh](#)]

Figure 170: Using ABS() and arithmetic operations in SELECT

ISBN	Quantity -13	ABS(QUANTITY -13)
12345	-3	3
54321	2	2
11111	-1	1
22222	-5	5
33333	1	1
44444	-7	7
55555	-4	4
66666	0	0
77777	-3	3
88888	-6	6

Figure 171: Result Table

```
SELECT ISBN, POWER(QUANTITY, 2), POWER(QUANTITY, 3)
FROM booktable
```

ISBN	POWER(QUANTITY, 2)	POWER(QUANTITY, 3)
12345	100	1000
54321	225	3375
11111	144	1728
22222	64	512
33333	196	2744
44444	36	216
55555	81	729
66666	169	2197
77777	100	1000
88888	49	343

Figure 172: Using POWER() to calculate square and cube of quantity

```
SELECT ISBN, QUANTITY, SQRT(QUANTITY)
FROM booktable
```

ISBN	QUANTITY	SQRT(QUANTITY)
12345	10	3.1622776601683795
54321	15	3.872983346207417
11111	12	3.4641016151377544
22222	8	2.8284271247461903
33333	14	3.7416573867739413
44444	6	2.449489742783178
55555	9	3
66666	13	3.605551275463989
77777	10	3.1622776601683795
88888	7	2.6457513110645907

Figure 173: Using SQRT() function on quantity values

```
SELECT ISBN, PRICE, ROUND(PRICE)
FROM booktable
```

ISBN	PRICE	ROUND(PRICE)
12345	670.00	670
54321	940.00	940
11111	750.00	750
22222	620.00	620
33333	850.00	850
44444	540.00	540
55555	780.00	780
66666	900.00	900
77777	690.00	690
88888	560.00	560

Figure 174: Using Sqrt() function on quantity values

```
SELECT ISBN, PRICE, CEILING(PRICE)
FROM booktable
```

ISBN	PRICE	CEILING(PRICE)
12345	670.00	670
54321	940.00	940
11111	750.00	750
22222	620.00	620
33333	850.00	850
44444	540.00	540
55555	780.00	780
66666	900.00	900
77777	690.00	690
88888	560.00	560

Figure 175: Applying ROUND() and CEILING() on price values

```
SELECT ISBN, PRICE, SQRT(CEILING(PRICE)) FROM booktable;
```

ISBN	PRICE	SQRT(CEILING(PRICE))
12345	670.00	25.88435821108957
54321	940.00	30.659419433511783
11111	750.00	27.386127875258307
22222	620.00	24.899799195977465
33333	850.00	29.154759474226502
44444	540.00	23.2379000772445
55555	780.00	27.92848008753788
66666	900.00	30
77777	690.00	26.267851073127396
88888	560.00	23.664319132398465

Figure 176: Using SQRT() with CEILING(PRICE) in nested functions

```
SELECT SQRT(PRICE), ROUND(PRICE), CEILING(PRICE), FLOOR(QUANTITY)
FROM booktable
```

SQRT(PRICE)	ROUND(PRICE)	CEILING(PRICE)	FLOOR(QUANTITY)
25.88435821108957	670	670	10
30.659419433511783	940	940	15
27.386127875258307	750	750	12
24.899799195977465	620	620	8
29.154759474226502	850	850	14
23.2379000772445	540	540	6
27.92848008753788	780	780	9
30	900	900	13
26.267851073127396	690	690	10
23.664319132398465	560	560	7

Figure 177: Applying multiple numeric functions on price and quantity

```
SELECT ISBN, SUBSTRING(TITLE, 3,4)
FROM booktable
```

ISBN	SUBSTRING(TITLE, 3,4)
12345	e Hu
54321	Song
11111	Kil
22222	84
33333	ide
44444	e Gr
55555	by D
66666	e Al
77777	fern
88888	e Fa

Figure 178: Extracting substring from book titles using SUBSTRING()

```
SELECT ISBN, UPPER(TITLE)
FROM booktable
```

ISBN	UPPER(TITLE)
12345	THE HUNGER GAME
54321	A SONG OF ICE AND FIRE
11111	TO KILL A MOCKINGBIRD
22222	1984
33333	PRIDE AND PREJUDICE
44444	THE GREAT GATSBY
55555	MOBY DICK
66666	THE ALCHEMIST
77777	INFERNO
88888	THE FAULT IN OUR STARS

Figure 179: Converting titles to uppercase using UPPER()

```
SELECT ISBN, REVERSE(TITLE)
FROM booktable
```

ISBN	REVERSE(TITLE)
12345	emaG regnuH ehT
54321	eriF dna ecl fo gnoS A
11111	dribgnikcoM a lliK oT
22222	4891
33333	ecidujerP dna edirP
44444	ybstaG taerG ehT
55555	kciD yboM
66666	tsimehclA ehT
77777	onrefnl
88888	sratS ruO ni tluaF ehT

Figure 180: Reversing book titles using REVERSE() function

```
SELECT ISBN, LENGTH(TITLE)
FROM booktable
```

ISBN	LENGTH(TITLE)
12345	15
54321	22
11111	21
22222	4
33333	19
44444	16
55555	9
66666	13
77777	7
88888	22

Figure 181: Getting length of titles using LENGTH() function

Lab 15: Date & Aggregate function

```
INSERT INTO BookTable (ISBN, Title, PurchaseDate)
VALUES
('54321', 'A Song of Ice and Fire', NOW());
```

✓ 1 row inserted. (Query took 0.0003 seconds.)

```
INSERT INTO BookTable (ISBN, Title, PurchaseDate) VALUES ('54321', 'A Song of Ice and Fire', NOW());
```

[\[Edit inline \]](#) [\[Edit \]](#) [\[Create PHP code \]](#)

Figure 182: Inserting current date into PurchaseDate using NOW() function

```
SELECT ISBN, TITLE, EXTRACT(YEAR FROM PurchaseDate) AS Year FROM
booktable
```

ISBN	TITLE	Year
12345	The Hunger Game	2023
54321	A Song of Ice and Fire	2024
11111	To Kill a Mockingbird	2024
22222	1984	2023
33333	Pride and Prejudice	2024
44444	The Great Gatsby	2024
55555	Moby Dick	2023
66666	The Alchemist	2024
77777	Inferno	2023
88888	The Fault in Our Stars	2024
54321	A Song of Ice and Fire	2025

Figure 183: Extracting year from PurchaseDate in SELECT statement

```
SELECT ISBN, TITLE, EXTRACT(YEAR FROM PurchaseDate), EXTRACT(MONTH
FROM PurchaseDate), EXTRACT(DAY FROM PurchaseDate) AS Year FROM
booktable
```

ISBN	TITLE	EXTRACT(YEAR FROM PurchaseDate)	EXTRACT(MONTH FROM PurchaseDate)	Year
12345	The Hunger Game	2023	9	10
54321	A Song of Ice and Fire	2024	6	1
11111	To Kill a Mockingbird	2024	3	15
22222	1984	2023	11	5
33333	Pride and Prejudice	2024	1	20
44444	The Great Gatsby	2024	4	12
55555	Moby Dick	2023	12	25
66666	The Alchemist	2024	2	18
77777	Inferno	2023	10	8
88888	The Fault in Our Stars	2024	5	1
54321	A Song of Ice and Fire	2025	6	5

Figure 184: Extracting year, month, and day from PurchaseDate

```
SELECT ISBN, TITLE, DATE_ADD(PurchaseDate, INTERVAL 5 DAY) AS NEWDATE
FROM booktable
```

ISBN	TITLE	NEWDATE
12345	The Hunger Game	2023-09-15 00:00:00
54321	A Song of Ice and Fire	2024-06-06 00:00:00
11111	To Kill a Mockingbird	2024-03-20 00:00:00
22222	1984	2023-11-10 00:00:00
33333	Pride and Prejudice	2024-01-25 00:00:00
44444	The Great Gatsby	2024-04-17 00:00:00
55555	Moby Dick	2023-12-30 00:00:00
66666	The Alchemist	2024-02-23 00:00:00
77777	Inferno	2023-10-13 00:00:00
88888	The Fault in Our Stars	2024-05-06 00:00:00
54321	A Song of Ice and Fire	2025-06-10 10:07:31

Figure 185: Adding 5 days to PurchaseDate using DATE_ADD()

```
SELECT ISBN, TITLE, DATE_SUB(PurchaseDate, INTERVAL 3 HOUR) AS NEWDATE
FROM booktable;
```

ISBN	TITLE	NEWDATE
12345	The Hunger Game	2023-09-09 21:00:00
54321	A Song of Ice and Fire	2024-05-31 21:00:00
11111	To Kill a Mockingbird	2024-03-14 21:00:00
22222	1984	2023-11-04 21:00:00
33333	Pride and Prejudice	2024-01-19 21:00:00
44444	The Great Gatsby	2024-04-11 21:00:00
55555	Moby Dick	2023-12-24 21:00:00
66666	The Alchemist	2024-02-17 21:00:00
77777	Inferno	2023-10-07 21:00:00
88888	The Fault in Our Stars	2024-04-30 21:00:00
54321	A Song of Ice and Fire	2025-06-05 07:07:31

Figure 186: Subtracting 3 hours from PurchaseDate using DATE_SUB()

TASK 2: AGREGATE

```
SELECT COUNT(*)
FROM booktable
```



Figure 187:Using COUNT(*) to count total records

```
SELECT SUM(QUANTITY) AS TOTAL
FROM booktable
```

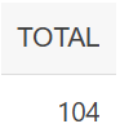


Figure 188: Using SUM() to calculate total quantity

```
SELECT MAX(QUANTITY) AS MAXIMUM  
FROM booktable
```

MAXIMUM
15

Figure 189: Using MAX() to find maximum quantity

```
SELECT COUNT(*), SUM(PRICE), MAX(QUANTITY), MIN(QUANTITY)  
FROM booktable
```

COUNT(*)	SUM(PRICE)	MAX(QUANTITY)	MIN(QUANTITY)
11	7300.00	15	6

Figure 190: Using aggregate functions: COUNT, SUM, MAX, and MIN in one query